ScintillaOS

操作系统内核设计文档



学校：武汉大学

队员：雷雨泽、杜宇恒、宋天琦

二〇二六年六月

摘　　要

ScintillaOS 是一个采用 Rust 语言编写、基于 rCore 基础系统的操作系统内核，面向全国大学生操作系统设计大赛的 QEMU 评测环境。ScintillaOS 支持基于 RISC-V 和 LoongArch 的平台适配，采用 RISC-V Linux 系统调用 ABI；系统实现了用户态 ELF 装载、Sv39 虚拟内存、进程与信号、文件描述符、管道、Ext4 文件系统、VirtIO 块设备以及 TCP/UDP 套接字等基础能力。

ScintillaOS 这一名称源于 Scintilla。Scintilla 有“火花”之意，寓意 **ScintillaOS 并没有选择现成的完整操作系统内核优化**，而是在 rCore 底层内核提供的基础火花之上，沿着启动、内存、进程、文件系统、设备驱动与网络支持等方向逐步延展、蔓延并搭建起来。这个名字既概括了 ScintillaOS 从核心出发不断扩展的开发过程，也表达了**团队希望能够从教学级基础内核开始，真正进行一次类 linux 操作系统内核的完整开发**，打磨出可运行、可扩展、可评测的竞赛内核的目标，从而深度锻炼团队的整体计算机系统工程能力。

本报告在 1-8 章具体说明了系统的总体架构和关键模块设计，并在第 9-10 章详细记录了本操作系统的开发流程，开发时所遇到的典型问题与团队所采用的具体解决方案。此外，本报告在附录中依照大赛要求提供了 AI 使用说明。截至 2026 年 6 月 29 日 12:00，ScintillaOS 在全国大学生操作系统设计大赛平台上取得了 1906 分，在排行榜上位列 47 名。

关键词: ScintillaOS；操作系统内核；Rust；RISC-V；Loongarch；rCore

评测完成

开始时间：2026-06-29 09:48:47.034337+08:00

结束时间：2026-06-29 11:56:02.227621+08:00

总分：1906.5102347009993

得分详情

测试点	glibc-la	glibc-rv	musl-la	musl-rv	总分
basic	102	102	102	102	408
busybox	54	54	54	54	216
cyclictest	0.0	0.0	0.0	0.0	0.0
iozone	20.0	20.0	20.0	20.0	80.0
iperf	0.0	0.0	0.0	0.0	0.0
libcbench	30.98880191481142	0.0	29.570856568964814	27.136759589774613	87.69641807355084
libctest	-	-	215	215	430
lmbench	45.41394127071955	45.06188963535785	46.335299714047146	46.53068761507888	183.34181823520342
ltp	1068	1068	1066	1066	4268
lua	9	9	9	9	36
netperf	0.0	0.0	0.0	0.0	0.0
总分	1329.402743185531	1298.0618896353578	1541.906156283012	1539.6674472048535	1906.5102347009993

图 1　ScintillaOS 的得分情况。ScintiallaOS 通过了 basic，busybox，iozone，libcbench，libctest，lmbench，ltp，lua 等多个官方测试用例。



图 2　ScintillaOS 的排名。截至 2026 年 6 月 29 日，ScintillaOS 在排行榜上位列 47 名。

目　录

1 系统概述 **1**

1.1 名称含义 1
1.2 设计目标与运行环境 1
1.3 总体架构 2
1.4 组件边界与设计取舍 3
1.5 模块协作机制与 Rust 语言特性 3
1.6 系统调用接口设计 4
1.7 代码组织 5
1.8 启动与执行流程 5
1.9 分工与贡献 5
1.10 参考与改进 6

2 内核关键设计与实现 **7**

2.1 rCore 基础 8
2.2 ScintillaOS: 从裸机启动到用户态执行 9
2.3 进程模型与调度：以内核对象组织用户程序生命周期 11
2.4 虚拟内存：同时支撑隔离、装载与按需分配 13
2.5 VFS 与文件描述符：用统一抽象连接磁盘、管道、设备和 proc . . 14
2.6 Linux 系统调用兼容：在 ABI 边界上保持语义一致 16
2.7 并发同步与资源管理：在单核前提下控制共享状态 17
2.8 平台与设备适配：在 QEMU virt 上验证完整内核路径 17
2.9 小结 18

3 内存管理模块 **19**

3.1 虚拟内存机制 19
3.2 物理内存分配 19
3.3 ELF 装载与用户栈 20
3.4 缺页、按需分页与写时复制 21
3.5 共享地址空间与堆管理 22

4 进程管理模块 **24**
4.1 核心数据结构 24
4.2 PID 管理与调度器 25
4.3 创建、切换与执行 26
4.4 进程层级、组与资源边界 26
4.5 进程生命周期 27
4.6 信号机制 27
4.7 退出、回收与等待 27

5 文件系统模块 **30**
5.1 虚拟文件系统架构 30
5.2 核心对象与对象关系 30
5.3 路径解析与文件打开 31
5.4 Ext4 与双盘设计 33
5.5 挂载、卸载与一致性 34
5.6 特殊文件和 IPC 35

6 信号处理模块 **36**
6.1 信号发送与待处理状态 36
6.2 信号递送时机 36
6.3 信号处理的限制 37

7 系统调用与用户态支持 **38**
7.1 系统调用接口 38
7.2 系统调用支持 39
7.3 用户态运行时 40

8 网络模块 **41**
8.1 网络栈架构 41
8.2 套接字与阻塞 I/O 42
8.3 限制与适配范围 42

9 开发目标、计划与过程记录 **43**
9.1 开发目标 43
9.2 阶段计划 44
9.3 完整开发过程描述 44
9.4 团队分工协作与证据 45

10 Scintilla 技术问题及解决方案示例 **47**

10.1 问题记录 47
10.2 案例一：VFS 与双盘路径解析 48
10.3 案例二：fork 与 COW 的一致性 50
10.4 案例三：系统调用边界条件 51

11 构建、评测与结果 **53**
11.1 构建与运行方法 53
11.2 测试框架 53

12 总结与展望 **54**
12.1 当前工作情况 54
12.2 经验总结 54
12.3 未来展望 54

附录 A AI 辅助使用说明 **55**

附录 B 参考资料 **57**

1 系统概述

1.1 名称含义

ScintillaOS 这一名称源于 Scintilla。Scintilla 有“火花”之意，寓意 ScintillaOS 并不是凭空生成的庞然整体，而是在 rCore 教学内核提供的基础火花之上，沿着启动、内存、进程、文件系统、设备驱动与网络支持等方向逐步延展、蔓延并搭建起来。这个名字既概括了 ScintillaOS 从核心执行链路出发不断扩展的开发过程，也表达了团队希望把一个教学内核的起点打磨成可运行、可测试、可继续演进的竞赛内核的目标。

1.2 设计目标与运行环境

ScintillaOS 的目标是在 QEMU 虚拟平台上提供兼容 RISC-V Linux ABI 的用户程序运行环境，并接入大赛测试盘完成基础功能、文件系统、网络与性能测试。内核主要使用 Rust 实现，用户程序为静态链接 ELF，运行时不依赖动态链接器。

ScintillaOS 内核主要面向单 CPU 环境进行设计，进程调度采用 FIFO 就绪队列配合时钟中断驱动的轮转策略。体系结构方面，当前以 RISC-V 作为主要评测目标，同时构建系统也支持 LoongArch 架构。在内存管理上，RISC-V 平台启用三级页表方案，并固定使用 4 KiB 的页面大小。文件系统采用 Ext4 格式，具体实现上通过 `lwext4_rust` 库来调用嵌入式 lwext4 的底层功能。内核则依赖 QEMU Virt 平台进行运行，并通过 OpenSBI 进行固件交互，外设方面支持 VirtIO 块设备和 VirtIO 网卡。在用户接口层面，内核遵循 RISC-V Linux ABI 的系统调用编号，所有系统调用均在陷阱处理流程中统一完成分发与处理。

表 1.1　系统主要特征

特征	说明
处理器模型	当前内核同步模型面向单 CPU，采用 FIFO 就绪队列和时钟中断驱动的轮转调度。
体系结构	RISC-V 为主要评测目标；构建系统还提供 LoongArch 目标。
地址空间	RISC-V 使用 Sv39 三级页表，页面大小为 4 KiB。
文件系统	Ext4；通过 `lwext4_rust` 调用嵌入式 lwext4 库。
设备与虚拟化	QEMU Virt 平台、OpenSBI、VirtIO 块设备和 VirtIO 网卡。
接口	系统调用编号遵循 RISC-V Linux ABI，内核在陷阱处理处完成分发。

1.3　总体架构

系统可分为用户态、系统调用与陷阱层、内核核心子系统以及设备层四个层次。图1.1概括了这些层次之间的调用关系：用户程序通过系统调用进入内核，内核再把请求分发给进程、内存、文件系统、信号和网络等子模块；设备层通过 VirtIO、串口和块设备为上层能力提供输入输出基础。

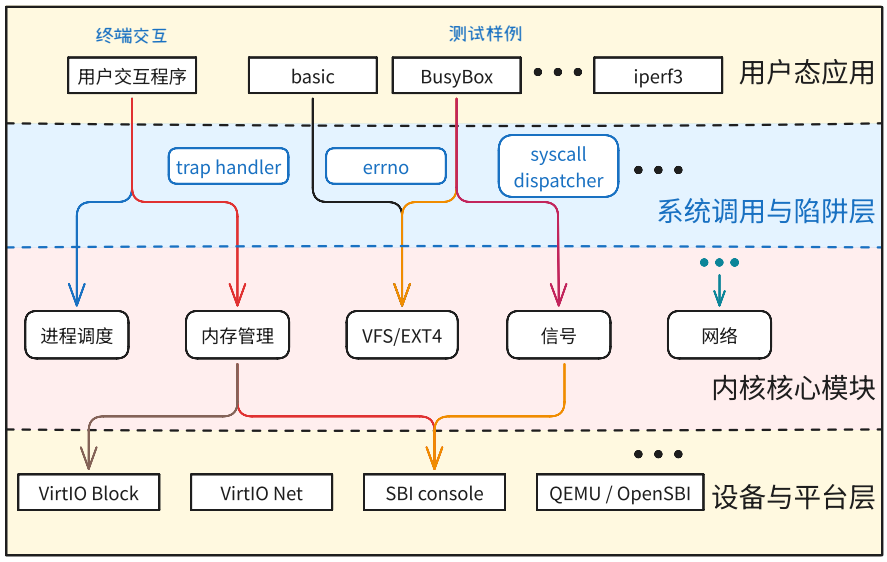


图 1.1　ScintillaOS 总体架构

1.4 组件边界与设计取舍

内核按功能划分模块，而非将全部逻辑集中在系统调用实现中。`task` 模块拥有进程状态和调度队列；`mm` 模块负责页表与地址空间；`fs` 模块将 VFS 接口下发给 Ext4；`net` 模块维护套接字及网络轮询；`drivers` 模块屏蔽 VirtIO 设备细节。系统调用层只承担参数转换、用户地址校验、权限检查和模块调用，从而使同一底层能力可以被多个系统调用复用。

ScintillaOS 目前以单处理器为目标。全局可变状态大多由 `UPSafeCell<T>` 封装：它基于 `RefCell` 提供内部可变性，并依赖单核和中断控制这一前提。因此该设计降低了单核 bring-up 的复杂度，但并不等价于多核可用的锁实现。报告中应明确该前提，避免将其表述为通用 SMP 同步方案。

1.5 模块协作机制与 Rust 语言特性

ScintillaOS 并非将进程、内存和文件系统孤立实现，而是围绕“任务运行一个用户程序”这一主线协作：用户态通过 `ecall` 进入内核；系统调用层校验用户参数后调用进程、内存、文件系统或网络模块；模块返回结果后由陷阱层写回寄存器；时钟中断则驱动调度、定时器和网络轮询。以 `fork` 为例，进程模块创建子 TCB，内存模块建立 COW 映射，文件系统模块增加 fd 对象引用，调度模块最后才将子任务加入就绪队列。任何一层遗漏都会使父子进程的资源语义不一致。

表 1.2 关键机制、引入原因与 Rust 支撑

机制	要解决的问题	设计选择	Rust 语言特性
COW 与按需分页	fork 立即复制大量页、文件映射提前读入会浪费内存	共享只读页，写缺页时复制；首次访问文件页时再装入	`FrameTracker` 的 `Drop` 回收、引用计数共享帧
TCB 与调度队列	需要统一保存上下文、fd、地址空间和亲缘关系	以 TCB 作为任务所有运行时状态的聚合点；FIFO 队列调度	`Arc` 共享任务对象、`Weak` 表示父引用
VFS 与 Dentry	多文件系统和挂载点不能依赖字符串拼接	超级块、Dentry、挂载表和 `File` trait 分层	trait 统一 I/O 对象；`Arc/Weak` 管理目录树生命周期
信号帧	用户处理函数执行后必须恢复被打断现场	在用户栈构造 `siginfo/ucontext`，由 `sigreturn` 恢复	结构体布局与边界检查限制不安全用户地址访问
socket fd	网络对象需与文件 I/O 统一管理	socket 创建后进入 fd 命名空间，`read/write` 可分派到网络层	枚举区分 TCP/UDP，所有权决定 close 后回收

表中 “Rust 支撑” 表示该语言特性如何降低资源管理错误的概率，而非宣称 Rust 自动保证内核正确：`UPSafeCell` 使用了 `unsafe impl Sync`，页表修改、用户指针转换和设备 DMA 仍须依赖代码审查与测试验证。

1.6 系统调用接口设计

用户程序将系统调用号放入 RISC-V 的 `a7` 寄存器，将最多六个参数放入参数寄存器后执行 `ecall`。我们通过陷阱处理程序识别用户态系统调用，调用 `syscall(syscall_id, [args; 6])`，并将返回值写入 TrapFrame 的返回寄存器。而从代码层面来看，文件、进程和网络调用则分别分派到 `syscall/fs.rs`、`syscall/process.rs` 与 `net/syscall.rs` 三个文件中。

系统调用处理上，我们先把用户态传进来的数据——比如字符串指针、结构体地址——转换成内核能安全访问的引用，这样后面具体实现就可以避开 ABI 细节。我们充分参考 linux “一切皆文件” 的设计准则，普通文件、管道、stdin/stdout 采用同一套 read/write 接口，并用 Arc<dyn File + Send + Sync> 装载文件描述

符。没做完的系统调用则直接返回-ENOSYS。

1.7 代码组织

ScintillaOS 的操作系统设计在 main 和 MachineTest 两个分支中。其中 main 分支用来开发，而 MachineTest 用来官网评测。此外，我们在 `documents` 目录下详细记录了 ScintillaOS 内核的开发流程。ScintillaOS 的具体结构则参考下表:

目录	职责
`os/src`	内核主体；包括启动、陷阱、进程、内存、文件系统、网络、驱动和系统调用。
`user/src`	用户态运行时、系统调用封装以及测试启动程序。
`lwext4_rust`	lwext4 的 Rust 绑定及块设备适配层。
`basic`	基础系统调用测试程序与构建脚本。
`documents`	架构、VFS、Ext4 迁移和各测试接入过程的内部开发记录，可行性分析与技术讨论。

1.8 启动与执行流程

内核入口位于 `os/src/main.rs`。启动后，系统依次初始化内核堆和物理帧分配器，挂载根 Ext4 文件系统，探测网络设备，创建内核页表上下文，并装载 `/bin/initproc` 进入就绪队列。调度器从 FIFO 队列中取出任务、切换页表和上下文；时钟中断触发网络轮询、定时器检查与调度。

这一启动路径决定了 ScintillaOS 的最小可验证闭环：内核不仅要能够被固件加载，还要完成根文件系统挂载、用户程序装载和第一次用户态返回。后续所有系统调用、文件访问和进程调度测试，都是在这一闭环稳定之后才具备意义。

1.9 分工与贡献

竞赛内核的功能往往跨越多个模块，因此分工不仅体现为代码目录的划分，也体现为接口设计、调试验证和文档整理之间的协作。表1.4按照模块边界整理了 ScintillaOS 的主要工作内容，便于说明每一类贡献对应的交付物和验证方式。

表 1.4　小组成员分工与可验证交付物

成员	主要模块	可验证交付物	证据
队员	内存管理	COW、缺页处理、ELF 装载、mmap 路径	源码记录 / 测试日志
队员	VFS/Ext4	Dentry、挂载、双盘访问、特殊文件	源码记录 / 文件系统测试
队员	进程与网络	fork/exec/wait、信号、socket、iperf3 路径	源码记录 / 回归日志

1.10　参考与改进

ScintillaOS 沿用教学操作系统中“陷阱—系统调用—进程—地址空间”的基本分层，同时针对大赛测试环境进行了工程化扩展：以 Linux RISC-V ABI 接入现有用户程序；从单一文件系统实例迁移到可挂载 VFS；使用双盘区分参赛者根文件系统和评测测试盘；针对 BusyBox、LTP、IOzone 与 iperf3 补齐调用与运行时路径。与 Chronix 等完整宏内核相比，ScintillaOS 明确以单核 QEMU 功能兼容为当前范围，不宣称已实现多核负载均衡、完整作业控制或完整 IPv6 网络栈。

2 内核关键设计与实现

本章围绕内核关键模块的设计与实现展开说明。需要首先明确的是，ScintillaOS 并不是从零开始实现一个操作系统内核，而是在 rCore 教学内核的基础上继续开发。rCore 已经提供了 RISC-V 平台上的基本裸机启动流程、特权级切换、异常与中断框架、任务抽象、地址空间管理以及 Rust 编写内核的基本工程结构。我们选择基于 rCore 开发，是因为它提供了一条清晰的最小内核主线：从机器启动、进入内核、建立页表，到创建用户任务并通过系统调用返回内核。这使 ScintillaOS 能够将主要精力放在比赛要求的 Linux 兼容能力、文件系统能力和复杂资源管理能力上，而不是重复搭建最基础的内核运行框架。

在此基础上，ScintillaOS 的技术路线可以概括为：继承 rCore 的 RISC-V 启动、陷阱处理、任务切换和虚拟内存框架，并围绕 Linux 应用兼容需求对其进行系统性扩展。首先，内核需要稳定支持异常、中断和系统调用，使用户程序能够通过受控入口请求内核服务；其次，需要以进程模型、地址空间和文件描述符表为资源组织核心，支撑 `fork`、`exec`、`wait` 等 Linux 进程语义；随后，需要进一步扩展虚拟内存、VFS 和设备抽象，使 `mmap`、Ext4、pipe、proc、VirtIO 块设备和网络设备等能力能够接入统一内核框架；最后，通过 Rust 的所有权、引用计数和受控的 `unsafe` 边界，降低页帧、文件对象、进程对象和设备缓冲区在复杂路径中的生命周期管理风险。

选择这一路线的原因在于，比赛内核的难点不只是“补充若干系统调用”，而是让已有 rCore 内核从教学型最小系统逐步扩展为能够运行更复杂 Linux 用户程序的兼容内核。系统调用、进程、内存、文件系统和设备驱动之间并不是相互独立的功能模块。例如，`fork` 依赖地址空间复制和文件描述符继承，`exec` 依赖 ELF 装载和页表重建，`mmap` 同时依赖 VFS 与缺页处理，Ext4 和 VirtIO 又决定用户程序能否访问真实磁盘数据。因此，本章后续内容将按照这些模块之间的依赖关系展开：先说明 rCore 提供的基础执行链路，再说明我们如何在此基础上扩展 Linux 兼容语义、文件系统、设备抽象和资源管理机制。

2.1 rCore 基础

rCore 为 ScintillaOS 提供的是一条已经打通的最小内核执行链路。它基于 RISC-V 架构和 Rust 语言实现，从固件交接、进入 S 态内核、建立栈和页表，到设置陷阱入口、创建用户任务、通过 `ecall` 返回内核，形成了操作系统从裸机启动到用户态执行的基本闭环。ScintillaOS 选择继承这条主线，是因为比赛内核的重点并不是再次证明“内核能够启动”，而是在已有执行链路上扩展 Linux 兼容语义、文件系统、设备和资源管理能力。

rCore 的价值首先体现在边界清晰。RISC-V 的特权级、异常中断和页表机制，使用户态和内核态之间存在明确的硬件隔离；rCore 在此基础上把系统调用、缺页异常、非法指令和时钟中断统一纳入陷阱处理路径。这样，ScintillaOS 后续实现 `fork`、`exec`、`mmap`、文件访问和设备中断时，不需要重新设计用户态进入内核的方式，而是可以围绕同一个可信入口扩展不同模块的语义。

rCore 的第二个价值在于资源抽象已经具备雏形。任务上下文、地址空间、页表映射、物理页帧分配和基础文件访问共同构成了进程运行的必要条件。ScintillaOS 在这些抽象上继续向 Linux 语义靠拢：任务需要扩展为包含父子关系、退出状态和文件描述符表的进程；地址空间需要支持 COW、文件映射和按需装页；文件访问需要从简单文件系统扩展为 VFS、Ext4、pipe、proc 和设备文件。也就是说，rCore 提供的是骨架，ScintillaOS 的工作是在骨架上补齐比赛所需的兼容能力。

采用 Rust 也是继承 rCore 的重要原因。页帧、进程对象、文件对象和目录项都具有复杂生命周期，Rust 的所有权、引用计数和 RAII 能够帮助内核把大部分资源释放逻辑收束到对象边界上。不过，陷阱入口、页表切换、用户指针访问和设备 MMIO 仍然必须使用 `unsafe`。因此，ScintillaOS 没有把 Rust 理解为自动安全保证，而是沿用 rCore 的思路：让普通内核逻辑尽量保持在安全抽象中，把必须越过类型系统的部分集中在硬件交互和地址转换边界。

因此，本节只概括 rCore 对 ScintillaOS 的基础作用，而不展开完整教学内核的演进过程。后续各节关注的重点，是 ScintillaOS 如何沿着 rCore 已经建立的启动、陷阱、进程、内存和文件访问链路，进一步扩展出可以运行更复杂 Linux 用户程序的内核能力。

2.2 ScintillaOS: 从裸机启动到用户态执行

对于 ScintillaOS 而言，内核启动阶段的核心任务，是把机器从固件交接后的运行状态，逐步转换为能够运行用户程序的环境。之所以首先实现启动流程、页表、陷阱入口和第一个用户进程，是因为后续所有系统调用、进程调度、文件访问和内存管理，都依赖这些基础设施。没有物理页帧管理，就无法为用户进程构造独立地址空间；没有根文件系统，就无法装载 `/bin/initproc`；没有陷阱入口，用户程序发生系统调用、缺页或非法访问时就无法安全回到内核。

因此，ScintillaOS 的启动路径采用如下设计：内核首先完成堆、帧分配器和页表初始化，使自身具备动态分配和地址转换能力；随后初始化陷阱向量，使异常、中断和系统调用都能进入统一入口；再挂载根文件系统并初始化 VirtIO 设备，使内核能够读取磁盘中的用户程序；最后装载 `/bin/initproc`，创建首个任务控制块，并通过上下文切换和 `sret` 进入用户态。

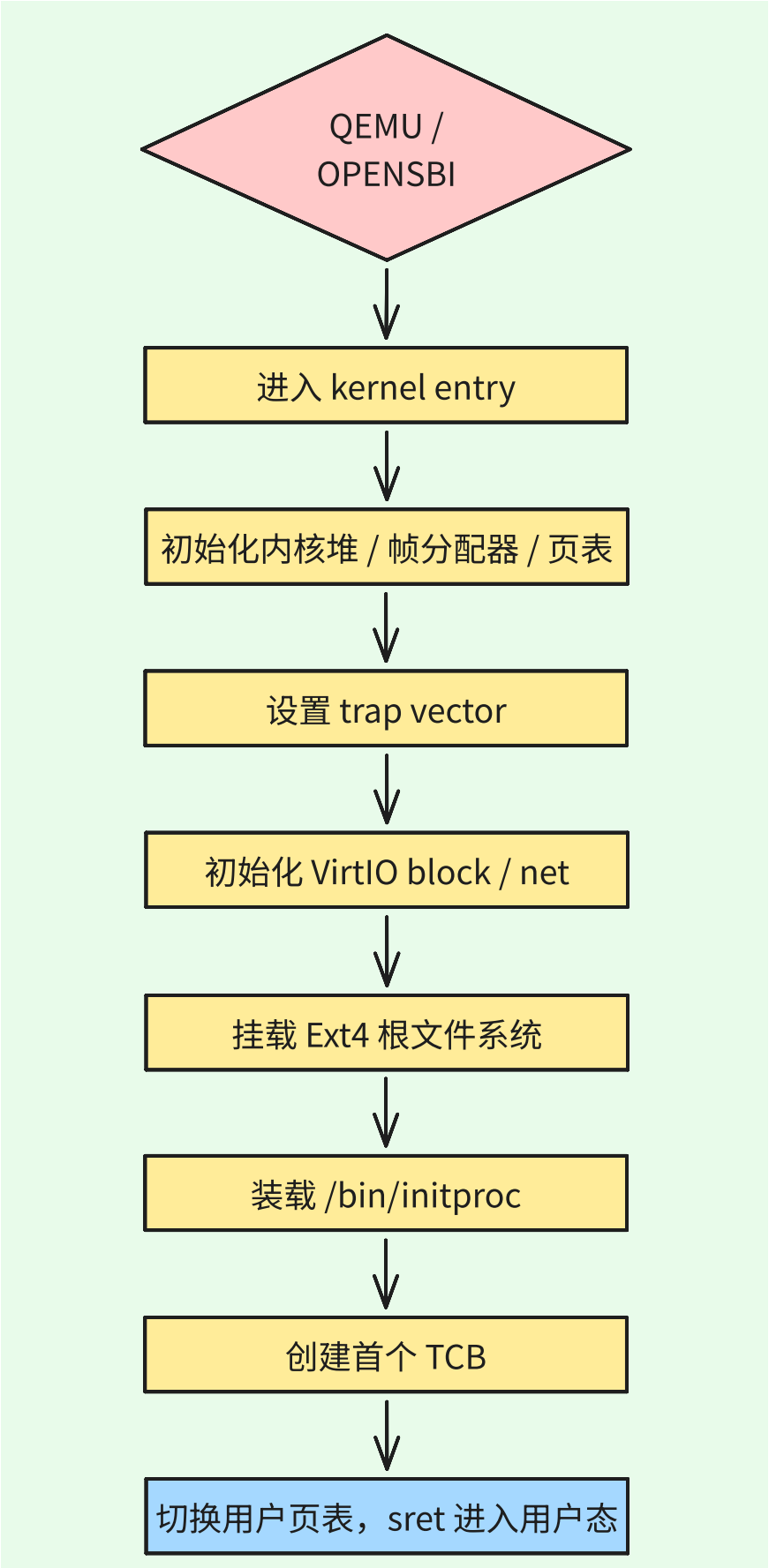


图 2.1　从内核启动到首个用户进程执行的主路径

统一陷阱入口是 ScintillaOS 后续设计的关键。用户程序不能直接调用内核函数，也不能直接访问内核资源，它只能通过硬件定义的异常入口进入内核。系统调用属于同步异常，用户程序按照 RISC-V Linux ABI 将系统调用号放入 `a7`，参数放入 `a0--a5`，执行 `ecall` 后进入内核。时钟中断和设备中断则是异步事件，可能在用户程序任意位置发生，因此内核必须保存被打断的寄存器现场，处理定时器、设备轮询或调度事件后，再恢复原任务或切换到新任务。

这里选择“统一陷阱入口 + 分发处理”的技术路线，是为了把用户态越权访问、缺页、系统调用和时钟抢占全部纳入同一个可信边界。这样做可以避免每个模块自行处理特权级切换，从而降低安全边界分散带来的错误风险。需要强调的是，Rust 能够减少普通控制流中的生命周期和资源管理错误，但陷阱入口、汇编保存现场、页表切换和用户指针访问仍然依赖架构约定与 `unsafe` 代码。因此，ScintillaOS 将 unsafe 尽量限制在硬件交互和边界转换位置，而不是让其扩散到普通业务逻辑中。

2.3 进程模型与调度：以内核对象组织用户程序生命周期

在完成从用户态进入内核的基本通路后，内核必须回答第二个问题：用户程序在内核中应当以什么形式存在。我们选择以“任务控制块 + 地址空间 + 文件描述符表 + 父子关系”作为进程抽象的核心，是因为 Linux 兼容语义中的 fork、exec、wait、exit、文件继承和资源回收，都需要围绕同一个进程对象展开。

`TaskControlBlock` 是进程资源的聚合点。调度器通过它找到可运行任务；系统调用通过它访问当前进程的地址空间、文件描述符表和工作目录；退出路径通过它保存退出码并通知父进程；wait 逻辑则通过父子关系回收僵尸进程。也就是说，TCB 并不是一个单纯的调度节点，而是连接调度、内存、文件和进程生命周期的中心对象。

当前调度器采用单核 FIFO 就绪队列。选择这一策略的原因是，ScintillaOS 主要面向 QEMU 单核评测环境，FIFO 调度实现简单、行为可预测，便于验证进程切换、阻塞唤醒和系统调用语义。它并不试图模拟 Linux CFS，也不解决 SMP 场景下的负载均衡问题。这样的取舍可以将实现重心放在 Linux ABI 兼容、地址空间隔离和文件系统正确性上，而不是过早引入复杂调度策略。

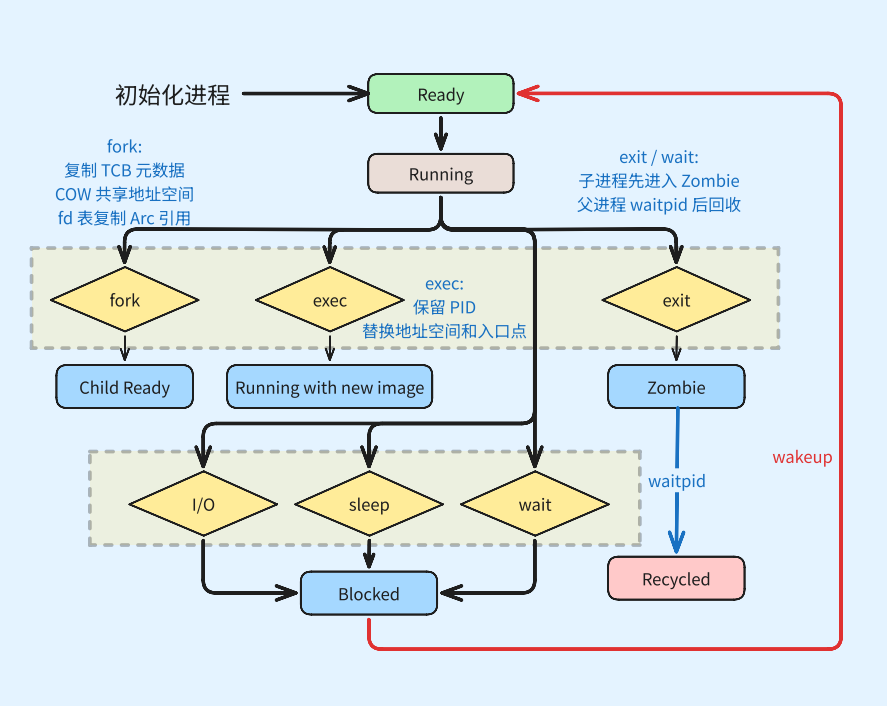


图 2.2　进程生命周期与资源语义

`fork`、`exec` 和 `wait` 体现了进程模型与其他模块之间的协作关系。`fork` 要求父子进程从同一个用户现场继续执行，但二者拥有不同返回值和独立进程身份。为了避免立即复制整个地址空间，内存模块为 fork 提供 COW 映射，使父子进程先共享只读物理页，只有发生写入时才复制页面。为了保持文件继承语义，文件描述符表复制的是 `Arc<dyn File>` 引用，而不是复制文件对象本身。这样父子进程可以共享打开文件状态，同时由引用计数保证对象生命周期。

`exec` 与 `fork` 的语义正好相反。它保留 PID 和部分进程属性，但用新的 ELF 映像替换当前地址空间，并重新设置用户栈和入口点。也就是说，exec 不是创建新进程，而是在原进程身份下运行新程序。`wait` 则对应进程退出后的回收阶段。子进程退出时不能立即释放所有元数据，否则父进程无法读取退出码；因此内核将其转入 Zombie 状态，等待父进程通过 wait 系列系统调用完成最终回收。

Rust 的 `Arc` 和 `Weak` 对这一设计非常重要。子进程对象可能同时被调度队列、父进程子表和其他内核路径引用，因此需要 `Arc` 保证对象不会被过早释放；父进程引用则适合使用 `Weak`，避免父子进程之间形成循环引用。文件对象、Dentry 和超级块等共享资源也通过 `Arc` 延长生命周期。与此同时，内核必须避免在阻塞或上下文切换前仍持有 TCB 内部状态的可变借用，否则即使在单核环境下，也可能

因为运行时借用冲突导致错误。

2.4 虚拟内存：同时支撑隔离、装载与按需分配

虚拟内存模块解决的是三个相互关联的问题：第一，不同用户进程之间必须相互隔离；第二，ELF 装载、用户栈、堆和 mmap 需要被组织在统一地址空间中；第三，fork 和文件映射不能在一开始就复制或读取所有数据，否则会带来明显的资源浪费。因此，ScintillaOS 以 `MemorySet` 表示进程地址空间，以 `MapArea` 表示连续虚拟区间，以页表项表示虚拟页到物理页的映射和权限。

这一路线的核心优势是把“地址区间语义”和“物理页实际分配”分离开来。`MapArea` 描述某段虚拟地址是否合法、具备什么权限、是否来自文件、是否支持 COW；页表项则描述当前是否已经建立到物理页的映射。这样，缺页异常就不再只是错误，而是内存模块实现按需分配、文件映射和 COW 的触发点。

表 2.1 页错误场景与处理方式

场景	判断依据	处理方式
文件映射首次访问	地址落在合法 `MapArea`，但尚未建立物理页映射	分配页帧，从文件读取对应内容，并建立页表映射
COW 写入	页表当前只读，但区域原本具有写语义，且物理页被共享	分配新页，复制旧页内容，更新当前进程页表并恢复写权限
非法访问	地址不属于用户空间，或访问权限与页表权限不匹配	返回错误、递送异常语义，或终止当前进程

选择这种设计，是因为它能够让 `fork`、`mmap` 和 ELF 装载共享同一套地址空间管理逻辑。fork 不需要专门复制全部物理页，只需要修改页表权限并标记 COW；mmap 不需要在调用时立即读取完整文件，可以在首次访问时按页装入；ELF 装载也可以通过 MapArea 管理代码段、数据段、用户栈和权限。这样既减少了重复代码，也使缺页处理成为内存模块的统一入口。

页表权限是用户态隔离的基础。读、写、执行和用户位不仅决定访问是否成功，也决定异常处理时内核如何解释错误原因。例如，用户程序向只读代码段写入，应被视为权限错误；对 COW 共享页写入，则应被解释为合法的写时复制；访

问未映射但属于合法文件映射区域的地址，则应触发按需装页。三者在硬件层面都可能表现为页错误，但在内核语义上完全不同，因此必须结合 `MapArea` 元数据和页表项状态共同判断。

Rust 的 RAII 机制适合管理物理页帧生命周期。`FrameTracker` 拥有物理页帧，在生命周期结束时通过 `Drop` 自动归还页帧；共享物理页则配合引用计数延迟释放。这种方式可以减少异常路径上的资源泄漏和重复释放问题。但页表项写入、地址翻译和用户缓冲区访问仍然是内核中最危险的部分，因为它们跨越了 Rust 类型系统与真实硬件状态。ScintillaOS 因此将这些操作集中封装，通过接口约束减少 unsafe 的扩散范围。

2.5 VFS 与文件描述符：用统一抽象连接磁盘、管道、设备和 proc

用户程序并不关心某个 fd 背后是普通文件、目录、管道、设备还是 proc 节点，它只期望通过 `open/read/write/close` 等接口完成访问。因此，文件系统模块需要解决的问题不是单纯读取 Ext4，而是将多种内核对象统一成“可以通过 fd 操作的资源”。这也是 ScintillaOS 引入 VFS、Dentry、挂载表和 `File` trait 的原因。

VFS 的设计目标是把路径解析、文件系统实例和打开文件对象分离。路径解析负责将 `/mnt/testcase` 这样的字符串逐级解析为目录项；Dentry 负责缓存路径分量与底层 inode 或挂载点之间的关系；超级块表示一个具体文件系统实例；`File` trait 则表示已经打开、可以被 fd 表引用的对象。这样，系统调用层不需要知道某个 fd 的底层类型，只需要调用统一的 read、write、stat 或 close 接口。

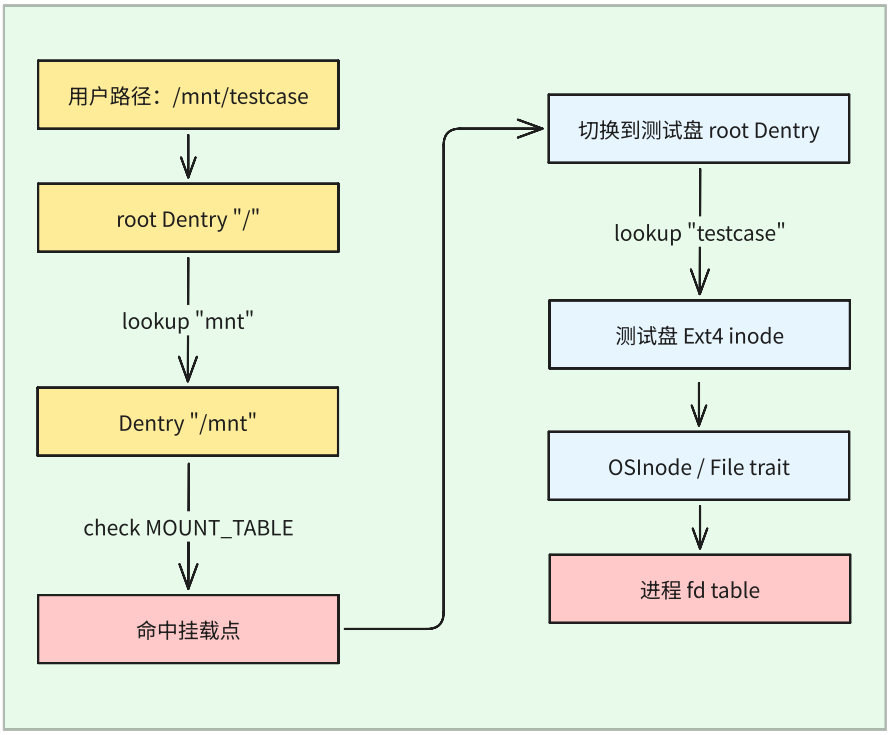


图 2.3　VFS 挂载点穿越流程

选择 VFS 路线的另一个原因，是 ScintillaOS 需要支持双盘和多类特殊文件。根文件系统和测试盘分别对应不同 VirtIO 块设备，`/mnt` 之后的路径应进入测试盘超级块，而不能继续访问根盘。如果仅把路径当作普通字符串拼接，就很难正确处理挂载点穿越、路径缓存和不同文件系统实例之间的切换。

pipe、设备文件和 proc 文件进一步说明了统一 fd 抽象的必要性。pipe 不对应磁盘 inode，而是内核中的环形缓冲区；`/dev/null` 和 `/dev/zero` 等设备文件由特殊逻辑生成读写行为；`/proc` 则把内核状态动态格式化为用户可读内容。它们的底层实现完全不同，但都可以实现 `File` trait 并进入 fd 表。这样，系统调用层不需要为每种资源维护一套单独的描述符机制，从而降低模块耦合。

文件系统模块也与进程和内存模块紧密相关。进程的 fd 表决定用户程序能访问哪些打开文件；fork 需要复制 fd 表并共享底层文件对象；exec 通常保留未设置 close-on-exec 的 fd；mmap 则需要从文件系统读取页面内容并交给内存模块建立映射。因此，VFS 不是孤立的文件访问层，而是连接进程生命周期、地址空间和设备驱动的重要中间层。

2.6 Linux 系统调用兼容：在 ABI 边界上保持语义一致

如果说陷阱入口解决的是“用户如何进入内核”，那么 Linux syscall ABI 兼容解决的就是“用户程序进入内核后期望看到什么行为”。系统调用兼容不能只理解为实现同名函数，它还包括寄存器传参、调用号、返回值符号、错误码、用户指针读取和各种边界条件。任何一个细节不一致，都可能导致 BusyBox、LTP 或比赛测例出现难以定位的异常。

在 RISC-V Linux ABI 中，系统调用号由 `a7` 传入，最多六个参数由 `a0--a5` 传入。内核需要在系统调用分发层完成参数读取，然后把语义处理转交给进程、文件、内存、网络等模块。成功时返回非负值，失败时返回负 errno。对于未实现的系统调用，返回 `-ENOSYS` 比直接伪造成功更安全，因为用户程序可能依赖该错误码进入降级路径，或者让测试明确暴露缺失能力。

表 2.2 系统调用实现时需要保持一致的语义点

维度	示例	报告中应说明的内容
ABI 参数	`openat` 四参数	参数从哪些寄存器取得，用户字符串和用户缓冲区如何通过当前页表安全读取
返回值	`read` / `write`	成功返回实际字节数，失败返回负 errno，短读短写不应被误判为内核错误
错误码	`ENOENT`、`EINVAL`、`ENOSYS`	路径不存在、参数非法、接口未实现等情况应返回不同错误码
边界条件	空路径、越界 fd、非法用户指针、非阻塞 I/O	不能只覆盖正常路径，还要说明异常输入、阻塞唤醒和权限检查

ScintillaOS 选择将系统调用层设计为“ABI 解码 + 模块转发”的薄层，而不是把所有逻辑堆在 syscall 文件中。这样做的原因是，系统调用本身只是用户态访问内核能力的入口，真正语义属于对应子模块。例如，`read` 的核心语义属于 fd 和 File trait；`fork` 属于进程和内存模块；`mmap` 同时涉及文件系统和虚拟内存；`waitpid` 则依赖进程父子关系和退出状态。保持这种分层可以让系统调用分发逻辑更清晰，也便于答辩时解释某个 syscall 的跨模块路径。

因此，系统调用支持状态不能只由 match 分支是否存在决定。一个 syscall 是

否真正可用，取决于其 ABI、主要语义、错误码和关键边界条件是否与测试期望一致。ScintillaOS 在描述支持程度时，需要同时结合源码路径和测试日志，区分“能够返回”“最小兼容”和“完整语义较一致”这几种不同层次。

2.7 并发同步与资源管理：在单核前提下控制共享状态

内核内部大量对象会被多个路径访问，例如任务队列、进程表、文件描述符表、页帧分配器、Dentry 缓存和设备队列。即使 ScintillaOS 主要面向单核 QEMU，也仍然需要清晰的同步策略。我们选择以 `UPSafeCell`、互斥封装和引用计数管理共享状态，是因为该方案与单核评测环境匹配，能够降低早期 bring-up 难度，同时保留明确的资源访问边界。

这一设计的前提是“单核 + 明确中断和调度点”。也就是说，系统不需要处理真正的多核并行访问，但必须处理任务在系统调用、阻塞、时钟中断和设备等待中的交错执行。因此，内核不能在持有 TCB 内部可变借用时主动阻塞，也不能在文件系统闭包和任务状态之间形成长时间嵌套借用，更不能在退出路径提前释放仍被线程或 fd 共享的对象。否则，即使没有 SMP 并发，也可能出现死锁、运行时借用冲突或资源悬空。

资源管理方面，ScintillaOS 尽量用 Rust 所有权表达内核对象生命周期。`FrameTracker` 管理物理页帧，`PidHandle` 管理 PID，`Arc<dyn File>` 管理 fd 指向的打开文件对象，`Arc` 和 `Weak` 管理进程树、Dentry 和超级块之间的引用关系。这样做的原因是，内核错误往往不是单个分支判断错误，而是资源在异常路径中释放过早、释放过晚、重复释放或形成循环引用。

不过，Rust 并不能自动证明整个内核安全。用户指针访问、页表修改、设备 MMIO、DMA 缓冲区、上下文切换和陷阱返回都需要突破普通类型系统。ScintillaOS 的实现原则是：普通业务逻辑尽量使用安全 Rust；必须 unsafe 的地方集中在硬件边界、地址转换和上下文切换附近，并通过接口封装限制其影响范围。这样可以让代码审查和测试更聚焦，避免 unsafe 逻辑在内核中无边界扩散。

2.8 平台与设备适配：在 QEMU virt 上验证完整内核路径

ScintillaOS 主要面向 QEMU RISC-V virt 平台，同时保留其他架构或构建目标的适配空间。选择 QEMU RISC-V virt 作为主要验证平台，是因为它提供了稳

定、可复现且与教学和比赛环境一致的硬件抽象，包括 OpenSBI 启动、RISC-V 特权级、SV39 页表、PLIC/CLINT 中断、VirtIO block、VirtIO net 和串口输出。通过固定内核镜像、根文件系统、测试盘和启动参数，团队可以反复复现实验结果，并利用串口日志定位启动、文件系统和系统调用错误。

平台适配本质上需要处理三类差异。第一是启动约定差异，不同架构或固件环境交给内核的入口状态、寄存器含义和内存布局可能不同。第二是异常与内存管理差异，包括陷阱寄存器、页表格式、地址空间标识和上下文保存方式。第三是设备与中断差异，例如 VirtIO 设备发现方式、中断号分配和网络后端行为。由于这些差异会直接影响内核控制流，ScintillaOS 将“存在构建目标”和“完成同等功能验证”分开描述，避免把某一平台上的测试通过结果直接泛化到另一平台。

QEMU 的价值不只是能够运行内核，还在于它让系统行为具备可重复性。对于操作系统开发而言，可复现环境与源码同样重要。提交材料中应记录 QEMU 版本、OpenSBI 或 RustSBI 版本、内存大小、磁盘镜像来源、网络参数和完整运行命令。这样，当文件系统挂载失败、VirtIO 初始化异常、网络测试不稳定或系统调用行为变化时，团队能够判断问题来自代码、镜像、平台参数还是外部环境差异。

2.9 小结

本章说明了 ScintillaOS 内核的核心技术路线：先通过启动流程和陷阱机制建立用户态与内核态之间的可信边界，再通过 TCB、地址空间和 fd 表组织进程资源，随后以虚拟内存、VFS 和系统调用分发支撑 Linux 兼容语义，最后通过 Rust 所有权、引用计数和受控 unsafe 降低资源管理风险。这些模块而是围绕同一个目标展开：我们充分参考 linux “一切皆文件”的思路，让用户程序能够以接近 Linux 的方式使用文件、内存、进程和设备，同时保证不同进程之间的隔离和内核资源生命周期的正确性。

3 内存管理模块

3.1 虚拟内存机制

内存管理首先要解决的是地址空间隔离问题。ScintillaOS 面向 RISC-V 平台时采用 Sv39 三级页表，虚拟页号被拆分为三级索引，页表项记录物理页号以及读、写、执行、用户访问等权限。选择这一路线的原因在于，Linux 用户程序默认相信自己运行在独立地址空间中；如果缺少页表隔离，进程之间的内存、用户态与内核态之间的边界都会失去硬件约束。

`MemorySet` 表示一个进程拥有的完整地址空间，内部保存页表和若干 `MapArea`。`MapArea` 描述连续虚拟区间的用途、权限、映射类型、已分配页帧以及可能的文件后备。这样设计的好处是把“某段地址是否合法”与“物理页是否已经分配”分开：ELF 段、用户栈、堆、匿名映射和文件映射都可以通过同一套区域操作完成建立、删除和拆分，缺页处理也能根据区域元数据判断是按需分配、COW 复制还是非法访问。

3.2 物理内存分配

地址空间管理最终必须落到物理页帧上。ScintillaOS 由 `frame_allocator.rs` 管理物理页帧，并使用伙伴系统分配器完成页帧申请与回收。选择伙伴分配的原因是页帧天然以固定大小对齐，分裂与合并逻辑清晰，适合内核早期建立页表、用户栈、COW 页面和文件缓存页面时反复申请整页内存。内核堆在启动阶段初始化，默认配置为 128 MiB，用于承载 `Arc` 对象、文件系统缓存和各类控制结构。

页帧不能只用裸物理地址表示，因为裸地址无法表达“谁负责释放”。ScintillaOS 使用 `FrameTracker` 以 RAII 方式封装页帧：对象创建时清空页面，复制共享页时增加引用计数，生命周期结束时自动归还或递减引用。代码3.1摘自 `os/src/mm/frame_allocator.rs:25--61`，展示了这一资源边界。

代码 3.1 `FrameTracker` 的构造、共享与释放

```
pub struct FrameTracker {
    pub paddr: PhysAddr, // 被跟踪物理页的起始地址
}
```

```
impl FrameTracker {
    pub fn new(paddr: PhysAddr) -> Self {
        paddr.clear_len(PAGE_SIZE); // 分配后先清空整页
        Self { paddr }
    }
    pub fn shared_clone(&self) -> Self {
        ref_inc(self.paddr); // 增加共享页引用计数
        Self { paddr: self.paddr }
    }
    pub fn is_shared(&self) -> bool {
        let refs = FRAME_REFS.lock();
        refs.get(&ref_key(self.paddr)).map_or(false, |&c| c > 1)
    }
}
impl Drop for FrameTracker {
    fn drop(&mut self) {
        let mut refs = FRAME_REFS.lock();
        let key = ref_key(self.paddr);
        if let Some(count) = refs.get_mut(&key) {
            // 仍有共享引用时只减少计数，不归还物理页
            if *count > 2 { *count -= 1; } else { refs.remove(&key); }
            return;
        }
        drop(refs);
        frame_dealloc(self.paddr); // 最后一个引用释放物理页
    }
}
```

`new` 在分配后清空整页，避免上一使用者的数据泄露到新映射；`shared_clone` 不复制物理页，而是增加引用计数，为 fork 的 COW 路径提供共享页面；`Drop` 则在对象离开生命周期时决定递减引用还是归还页帧。这样，页帧回收不依赖每条异常路径手工调用释放函数，而是与对象所有权绑定。对于内核而言，这一点很重要，因为进程退出、exec 替换、mmap 解除映射和 COW 拆分都可能改变同一物理页的所有关系。

3.3 ELF 装载与用户栈

ELF 装载解决的是“磁盘文件如何变成用户进程”的问题。原生 rCore 在 glibc 版本测试不能通过。我们思考发现，ScintillaOS 不能把用户程序简单

复制到某个固定地址执行，而必须根据 ELF 程序头建立带权限的地址空间。`MemorySet::from_elf` 解析 `PT_LOAD` 段，为代码、只读数据、可写数据和 BSS 建立映射，并构造用户栈。对于未页对齐的 ELF 段，装载过程保留段首页偏移，保证数据写入正确的虚拟地址；这对 musl 静态程序尤其重要，因为其装载段不一定天然按页对齐。

装载结果不仅包括页表映射，还包括入口地址、用户栈顶、程序头地址等启动信息，这些信息会交给进程创建逻辑写入 TrapFrame。典型用户地址空间由代码段、只读数据段、可写数据/BSS 段、用户栈、堆和 mmap 区域组成。用户栈默认大小为 256 KiB；用户运行时在 `_start` 阶段初始化堆，再解析参数并进入 `main`。因此，ELF 装载同时依赖文件系统、页表和进程上下文，是内存模块连接用户态程序的关键入口。

3.4 缺页、按需分页与写时复制

缺页异常在 ScintillaOS 中不只是错误路径，也是延迟分配资源的触发点。我们在测试 `lmbench` 测试用例时发现，如果按照 rCore 传统 fork 逐页复制父进程，会使创建成本随地址空间线性增长，而许多子进程随后会立即 `exec` 或只读取继承页面。因此 ScintillaOS 将 COW 作为 fork 的默认机制：父子进程最初共享只读物理页，只有某一方写入时才通过写缺页复制页面并恢复写权限。文件映射也采用类似思路，首次访问时才申请页帧并从文件填充数据。

ScintillaOS 通过 `mmap`、`munmap` 和 `msync` 分别负责建立映射、解除映射和脏页回写。这里需要区分两种“共享”：COW 的共享只是复制前的临时状态，写入时应拆分为私有页；`MAP_SHARED` 的共享映射则允许多个映射观察同一物理页或同一文件后备。代码3.2根据 `os/src/mm/memory_set.rs:516--561` 整理了复制用户地址空间的关键路径。

代码 3.2　从父地址空间创建子地址空间的 COW 路径

```
pub fn from_existed_user(user_space: &MemorySet) -> Option<MemorySet> {
    let mut memory_set = Self::new_bare();
    for area in user_space.areas.iter() {
        let mut new_area = MapArea::from_another(area); // 复制映射元数据
        for (&vpn, frame) in area.data_frames.iter() {
            let mut child_perm = area.map_perm;
            let new_frame = {
```

```
8                 let new_frame = frame.shared_clone(); // 父子先共享同一物理
                      帧
9                 if !area.shared && child_perm.contains(MapPermission::W) {
10                    child_perm.remove(MapPermission::W); // 首次写入时再触发
                          COW
11                    user_space.page_table.unmap_page(vpn);
12                    user_space.page_table.map_page(
13                        vpn, frame.paddr, child_perm.into(),
14                        MappingSize::Page4KB);
15                }
16                new_frame
17            };
18            // 在子页表中建立同一物理页的只读映射
19            memory_set.page_table.map_page(
20                vpn, new_frame.paddr, child_perm.into(),
21                MappingSize::Page4KB);
22        }
23    }
24    Some(memory_set)
25 }
```

这段逻辑先以 MapArea 为粒度复制映射元数据，再以虚拟页为粒度处理已经存在的物理帧。对非共享且原本可写的页面，内核移除写权限，并让父子双方引用同一个 FrameTracker。只有写缺页发生时，缺页处理才分配新帧并复制内容。这样，fork 的主要成本从“复制整个地址空间”变为“复制页表元数据和后续真正写入的页面”。

因此，COW 与 mmap 的正确性必须同时观察父子写入隔离、共享页引用计数、文件映射首次访问和解除映射后的资源回收。它们表面上都由缺页异常触发，但背后依赖的语义不同；只有把 MapArea 元数据、页表权限和页帧生命周期放在一起判断，才能避免把合法缺页误处理成进程错误，或把非法访问错误地当成按需分配。

3.5 共享地址空间与堆管理

fork 与 clone(CLONE_VM) 的区别，体现了 ScintillaOS 对地址空间所有权的不同处理。传统 fork 创建新的页表和映射元数据，并通过 COW 延迟实际页面复制；clone(CLONE_VM) 更接近线程式任务，子任务共享父任务的页表 Arc，因此退

出时不能提前回收仍被其他任务使用的页帧。这个差异决定了进程管理模块在创建、退出和回收时必须询问内存模块的共享状态。

`brk` 和 `mmap` 则分别对应两种用户态内存增长方式。`brk` 维护连续堆区末端，适合传统堆分配器；`mmap` 建立离散的匿名或文件映射，适合大块内存、文件映射和特殊运行时需求。`munmap` 需要处理完整删除、头部裁剪、尾部裁剪和中间分裂等情况，因此边界判断比普通释放页帧更复杂。ScintillaOS 将这些操作统一落到 `MapArea` 的区间维护上，是为了让堆、文件映射和缺页处理共享同一套地址空间语义。

4 进程管理模块

4.1 核心数据结构

进程管理要解决的是“一个正在运行的用户程序在内核中如何被表示”。与其他操作系统相同，ScintillaOS 采用 `TaskControlBlockInner`(TCB)。这样设计的原因在于，Linux 进程语义本身就是跨资源的：`fork` 需要同时复制地址空间和 fd 表，`exec` 需要替换地址空间但保留部分 fd，`waitpid` 又需要读取退出状态并回收子进程对象。

在此基础上，`TASK_MANAGER` 维护 FIFO 就绪队列，`PID2TCB` 负责按 PID 查找进程控制块。调度器并不直接理解文件系统或内存细节，而是通过 TCB 拿到可以切换的任务对象；系统调用层也不直接保存进程资源，而是通过当前 TCB 访问地址空间、cwd、root 和 fd 表。也就是说，TCB 是进程生命周期的中心节点，其他模块围绕它建立引用关系。

字段类别	作用
TrapFrame 与任务上下文	保存用户寄存器、内核栈指针和恢复执行所需上下文。
`memory_set`	指向任务虚拟地址空间；`CLONE_VM` 任务可共享该对象。
`fd_table`	文件描述符到 `File` trait 对象的映射，支持复制和 close-on-exec 处理。
`cwd/root`	当前目录和进程根目录的 Dentry 引用，用于相对路径和路径隔离语义。
父子关系	父进程弱引用及子进程表；退出时用于收养、wait 和 SIGCHLD。
信号状态	待处理位图、阻塞掩码、信号动作表、冻结/终止标志。
凭证与时间	uid/euid/gid/egid、用户态和内核态累计时间。

代码4.1给出了 fork 路径的关键摘录。该函数刻意分两个阶段读取父任务状态：先复制地址空间，再短时间借用父 TCB 收集其余状态，避免长时间持有内部可变借用。

代码 4.1　TaskControlBlock::fork 的关键资源复制逻辑

```
1  pub fn fork(self: &Arc<TaskControlBlock>) -> Option<Arc<TaskControlBlock
       >> {
2      let memory_set = {
3          let parent = self.inner_exclusive_access();
4          MemorySet::from_existed_user(&parent.memory_set)?
5      };
6      let (trap_cx, fd_table, cwd, root, signal_mask, actions) = {
7          let parent = self.inner_exclusive_access();
8          (parent.trap_cx.clone(),
9           parent.fd_table.iter().map(|f| f.as_ref().map(|x| {
10              x.dup_ref(); x.clone()
11          })).collect(),
12          parent.cwd.clone(), parent.root.clone(),
13          parent.signal_mask, parent.signal_actions.clone())
14     };
15     // 分配新的 PID 和内核栈，再构造子任务控制块。
16 }
```

第 3–5 行调用 MemorySet::from_existed_user 建立子地址空间，这里进入的是 COW 路径，而不是立即复制所有用户页。第 7–14 行复制用户现场、当前工作目录、根目录、信号掩码和动作表。文件描述符对象以 Arc 共享，因此复制 fd 表时增加底层对象引用，而不是复制文件内容。随后子任务分配新的 PID 和内核栈，并把子任务返回值置为 0；父任务则从系统调用返回子 PID。这个差异正是 Linux fork 语义中父子执行路径分叉的来源。

4.2　PID 管理与调度器

PID 是用户态观察进程的稳定身份，因此它的分配和释放必须与进程对象生命周期绑定。ScintillaOS 使用 PidHandle 管理 PID，在对象释放时自动归还编号，避免异常退出路径遗漏回收。调度方面，就绪队列使用 VecDeque，add_task 将可运行任务放入队尾，fetch_task 从队首取出任务。这一 FIFO 策略简单且可预测，适合单核 QEMU 环境下验证进程切换、阻塞唤醒和系统调用语义。

选择 FIFO 并不意味着忽略阻塞语义。任务可能因 nanosleep、网络 I/O、waitpid 或 futex 等原因离开就绪队列，对应的睡眠队列、网络阻塞队列、等待子进程队列和 futex 队列保存等待者；事件发生后，内核再把任务放回就绪队列。这里的关键约束是：阻塞前必须释放 TCB、文件系统或网络对象上的内部借用，否则任务切换后再次进入相关路径时，单核 UPSafeCell 模型也可能出现可重入借用错误。

4.3 创建、切换与执行

进程创建和调度的主线可以概括为如下过程：

fork/clone 创建 TCB → 分配 PID、内核栈和 TrapFrame →
复制或共享地址空间与 fd 表 → add_task 入就绪队列 →
fetch_task 取队首任务 → 切换页表和任务上下文 → 返回用户态

fork、clone 和 exec 都与新执行流有关，但它们的资源语义并不相同。fork 创建新的进程身份和独立页表，并通过 COW 延迟复制物理页；clone 根据标志选择是否共享地址空间、信号动作或其他资源；exec 不创建新 PID，而是在原进程身份下用新的 ELF 映像替换地址空间和 TrapFrame，并保留未设置 FD_CLOEXEC 的文件描述符。这样划分可以解释为什么进程模块必须同时依赖内存、文件和系统调用 ABI。

4.4 进程层级、组与资源边界

父子关系是 waitpid、孤儿进程收养和 SIGCHLD 递送的基础。ScintillaOS 通过 TCB 中的父引用和子表维护进程层级；父进程退出后，子任务交由 init 进程收养，使后续退出状态仍然有明确回收者。工程中定义了 setpgid、getpgid 和 setsid 的入口，但进程组、会话和作业控制并不是当前兼容能力的重点，相关行为需要以实际测试覆盖为边界。

资源边界则用于防止异常程序无限占用内核对象。fd 表设置最大描述符数量，用户栈和内核堆也采用固定配置。它们不是 Linux 通用限制，而是 ScintillaOS 在当前评测环境中的工程边界。报告中明确这一点，是为了避免把“足以支撑当前测试”的配置误写成完整资源隔离机制。

4.5 进程生命周期

进程生命周期可以看成一条从创建、执行、替换到退出回收的状态链。`fork` 复制进程状态并使用写时复制地址空间；`clone(CLONE_VM)` 可共享页表；`exec` 重新装载 ELF，同时保留未设置 close-on-exec 的文件描述符；`exit` 使进程停止运行并进入 Zombie 状态。这样设计的原因在于，父进程需要在子进程退出后读取退出码，内核不能在子进程停止执行的瞬间立即释放所有元数据。

4.6 信号机制

信号机制使进程能够接收异步事件，例如父子退出通知、用户终止请求或非法访问导致的致命事件。ScintillaOS 将待处理信号以位图形式保存在进程控制块中，并在陷阱返回用户态前统一处理。内核实现 `kill`、`tgkill`、`sigaction`、`sigprocmask` 和 `sigreturn` 等接口；默认动作包括终止、停止、继续或忽略。安装用户处理函数时，内核备份 TrapFrame，将 PC 改为处理函数入口，并在 `sigreturn` 中恢复原上下文。

4.7 退出、回收与等待

退出路径是进程管理中最容易跨模块出错的部分。当前任务调用 `exit` 或因致命信号退出时，内核需要同时更新进程状态、关闭文件描述符、回收非共享地址空间、处理孤儿进程并通知父进程。若父进程正在等待子进程，内核唤醒等待队列；若 `SIGCHLD` 未被屏蔽，则在父进程待处理信号集中置位。`waitpid` 最终读取退出码，并完成僵尸进程对象的回收。

因此，“退出”和“回收”必须区分为两个阶段：子进程先停止执行并保留退出状态，父进程随后通过 wait 类调用取得状态并完成最终回收。这个划分避免父进程尚未读取退出信息时丢失子进程状态，也让信号递送和等待队列唤醒有明确的同步点。

代码4.2根据 `os/src/task/mod.rs:384--451` 整理了退出路径。它展示了退出操作不仅是设置返回码，还必须同步处理地址空间、文件描述符、亲缘关系和等待者。

代码 4.2　当前任务退出与僵尸化处理

```
1 pub fn exit_current_and_run_next(exit_code: i32) -> ! {
```

```
    crate::fs::stdio::stdout_release_if_owner(); // 释放当前进程的终端输出所有权
    let task = current_task().unwrap();
    remove_from_ready_queue(task.getpid());
    remove_from_pid2task(task.getpid());
    let (parent, orphan_children) = {
        let mut inner = task.inner_exclusive_access();
        inner.task_status = TaskStatus::Zombie; // 保留退出状态，等待父进程回收
        inner.exit_code = exit_code;
        let parent = inner.parent.as_ref().and_then(|w| w.upgrade());
        let orphan_children = core::mem::take(&mut inner.children);
        inner.memory_set.recycle_data_pages(); // 回收非共享用户页
        inner.clear_fd_table(); // 关闭所有文件描述符
        (parent, orphan_children)
    };
    if let Some(parent) = parent {
        notify_parent_sigchld(&parent); // 通知父进程存在已退出子进程
        let mut queue = WAITPID_QUEUE.exclusive_access();
        if let Some(idx) = queue.iter().position(|t| t.getpid() == parent.getpid()) {
            let parent_task = queue.remove(idx);
            drop(queue);
            parent_task.inner_exclusive_access().task_status = TaskStatus::Ready;
            add_task(parent_task);
        }
    }
    if !orphan_children.is_empty() {
        let mut init = INITPROC.inner_exclusive_access();
        for child in orphan_children.iter() {
            // 孤儿进程由 init 进程收养
            child.inner_exclusive_access().parent = Some(Arc::downgrade(&INITPROC));
            init.children.push(child.clone());
        }
    }
    if let Some(current) = take_current_task() {
        defer_drop_task(current);
    }
    loop { schedule_idle_context(); }
}
```

实现首先取得当前 TCB，并把任务状态置为 `Zombie`；随后回收不与线程共享的用户页，关闭 fd 表。父子关系更新与 `SIGCHLD` 递送保证父进程能够通过 `waitpid` 观察退出状态。对于等待子进程或 futex 的任务，唤醒逻辑必须在状态更新之后执行，否则父进程可能被唤醒却看不到一致的僵尸状态。由此可见，退出路径不是单独的进程逻辑，而是进程、内存、文件和信号机制共同维护的一致性边界。

5 文件系统模块

5.1 虚拟文件系统架构

文件系统模块要解决的核心问题，是让用户程序通过统一路径和 fd 接口访问不同来源的对象。早期直接围绕单一 Ext4 实例和字符串路径实现文件访问，无法表达挂载点，也难以支持多个文件系统。例如测试盘挂载到 `/mnt` 后，`/mnt/a` 应访问另一块设备，而不是根盘目录。因此 ScintillaOS 引入 VFS 子层，用 `VfsSuperBlock` 表示已挂载文件系统实例，用 `Dentry` 表示路径分量，用 `Mount` 记录挂载点和被挂载根节点的对应关系。

VFS 的意义在于把“路径解析”和“具体 Ext4 调用”分离开来。路径解析逐级调用 `vfs_lookup`，命中 Dentry 缓存时直接返回，未命中时读取 Ext4 目录项并填充缓存；当路径进入挂载点时，解析过程透明切换到被挂载文件系统的根 Dentry。这样，`cwd`、`root` 和文件描述符引用的是稳定内核对象，而不是容易失效的字符串路径。

5.2 核心对象与对象关系

对象	设计与职责
`VfsSuperBlock`	一个已挂载文件系统的实例，内部保存文件系统类型和 Ext4 私有对象；由 `Arc` 共享。
`Dentry`	名字到文件系统对象的缓存节点，保存 `name`、统一命名空间路径、文件系统内部路径、父节点、子节点缓存和所属超级块。
`Mount`	将父文件系统中的挂载点 Dentry 映射到新文件系统根 Dentry。
`OSInode`	一个打开文件句柄，保存可读/可写标记、Dentry 和当前偏移；承担 ScintillaOS 中 file/inode 的主要运行时职责。

对象	设计与职责
File	统一 I/O trait。普通文件、Pipe、Stdin、Stdout 和部分特殊文件以同一方式进入 fd 表。

表中的对象共同构成 VFS 的分层边界：超级块负责文件系统实例，Dentry 负责命名空间节点，Mount 负责跨文件系统跳转，File trait 负责打开后的统一 I/O 行为。代码5.1是 os/src/fs/vfs.rs:545 中 Dentry 的结构摘录。它同时保存面向用户的统一路径 path 与交给 Ext4 的内部路径 fs_path；二者在根文件系统中通常一致，但在 /mnt 等挂载点之后会不同。

代码 5.1　Dentry 的关键字段

```
1  pub struct Dentry {
2      pub name: String,
3      pub path: String, // 统一命名空间中的路径
4      pub fs_path: String, // 所属文件系统内部路径
5      pub inode_num: u32,
6      pub is_dir: bool,
7      pub is_link: bool,
8      pub parent: Option<Weak<Dentry>>, // 使用弱引用避免目录树循环引用
9      pub super_block: Arc<VfsSuperBlock>, // 决定底层 Ext4 实例
10     pub children: UPSafeCell<Option<Vec<Arc<Dentry>>>>, // 惰性子目录缓存
11 }
```

第 2–4 行将“名字”“全局路径”和“文件系统内部路径”分离，避免把挂载逻辑退化为字符串拼接。第 7 行使用弱引用指向父节点，避免 Dentry 树形成引用环；第 8 行决定每次 I/O 应下发到哪个 Ext4 实例；第 9 行的 None 表示子目录尚未从磁盘装入，Some 则保存已缓存的子节点。该缓存主要服务于路径查找，并不等同于完整页缓存。

5.3　路径解析与文件打开

路径解析之所以需要单独说明，是因为它决定了用户命名空间如何映射到底层磁盘对象。绝对路径从任务的 root 开始，相对路径从 cwd 开始；解析器逐项处理“.”、“..”和普通路径分量。其中“..”必须感知挂载根，避免从被挂载文件系统根节点直接跳到错误父节点；普通分量则先查询 Dentry.children 缓存，未命中才

调用 Ext4 获取目录项并建立缓存。

代码10.2根据 os/src/fs/vfs.rs:688--735 整理。它说明路径解析并非简单拼接字符串：每一个路径分量都可能命中缓存、触发底层目录读取，或因为自身是挂载点而跳转到另一个超级块。

代码 5.2　单级 Dentry 查找和挂载点穿越

```
pub fn vfs_lookup(parent: &Arc<Dentry>, name: &str) -> Option<Arc<Dentry
    >> {
    match name {
        "." => return Some(parent.clone()), // 当前目录
        ".." => return vfs_parent(parent).or_else(|| Some(parent.clone()))
            , // 父目录
        _ => {}
    }
    {
        let children = parent.children.exclusive_access(); // 优先查询
            Dentry 缓存
        if let Some(list) = children.as_ref() {
            if let Some(cached) = list.iter().find(|c| c.name == name) {
                if let Some(mnt_root) = lookup_mount(cached) {
                    return Some(mnt_root);
                }
                return Some(cached.clone());
            }
        }
    }
    let child_fs_path = if parent.fs_path == "/" {
        format!("/{}", name)
    } else if parent.fs_path.ends_with('/') {
        format!("{}{}", parent.fs_path, name)
    } else {
        format!("{}/{}", parent.fs_path, name)
    };
    let info = parent.super_block.get_inode_info(&child_fs_path)?; // 缓存
        未命中，访问 Ext4
    let child = Dentry::new_child(parent, name, info.inode_num,
                                  info.is_dir, info.is_link);
    cache_child(parent, child.clone()); // 将新节点放入缓存
    if let Some(mnt_root) = lookup_mount(&child) {
        return Some(mnt_root); // 挂载点透明跳转到新文件系统根目录
    }
```

```
32      Some(child)
33  }
```

对于 “.”，查找直接返回当前节点；对于 “..”，实现转交给感知挂载关系的父目录逻辑。普通名称先查询 `children` 缓存，若缓存未初始化，再通过 `super_block` 调用 Ext4 读取目录项。查到节点后还需检查它是否为 `MOUNT_TABLE` 中的挂载点：如果是，返回挂载文件系统根目录。因此，用户对 `/mnt/foo` 的 `openat` 不需要知道 `/mnt` 前后对应不同磁盘。

`openat` 最终调用 `open_file_new`。带 `O_CREATE` 标志时，内核先检查目标是否存在，再决定打开、截断或创建；常规打开则解析路径并创建 `OSInode`。`/proc` 和 `/dev` 前缀在进入 Ext4 查询之前由专用逻辑截获，因此它们不依赖磁盘中的同名文件。这样，普通文件、特殊文件和挂载文件系统都能进入同一 fd 表。

5.4 Ext4 与双盘设计

Ext4 负责提供真实磁盘文件语义，VFS 负责把这些语义纳入统一命名空间。ScintillaOS 的底层 Ext4 功能由 `lwext4_rust` 提供，VFS 通过 `with_fs` 闭包在受保护的 Ext4 实例上执行读写、目录枚举、创建、截断和同步操作。块设备层将 VirtIO 块设备包装为连续数据流，再交给 lwext4 使用。这样的分层使文件系统逻辑不需要直接处理 VirtIO 队列细节。

双盘设计来自比赛评测需求。bus0 对应评测提供的测试盘，bus1 对应根文件系统镜像；启动时 `mount_root` 使用根盘，`sys_mount` 可将测试盘挂载到目标目录。这样可以避免测试数据与系统根文件混用，也使 `/mnt` 下的访问路径与评测场景保持一致。图5.1展示了用户命名空间、超级块和 VirtIO 块设备之间的关系。

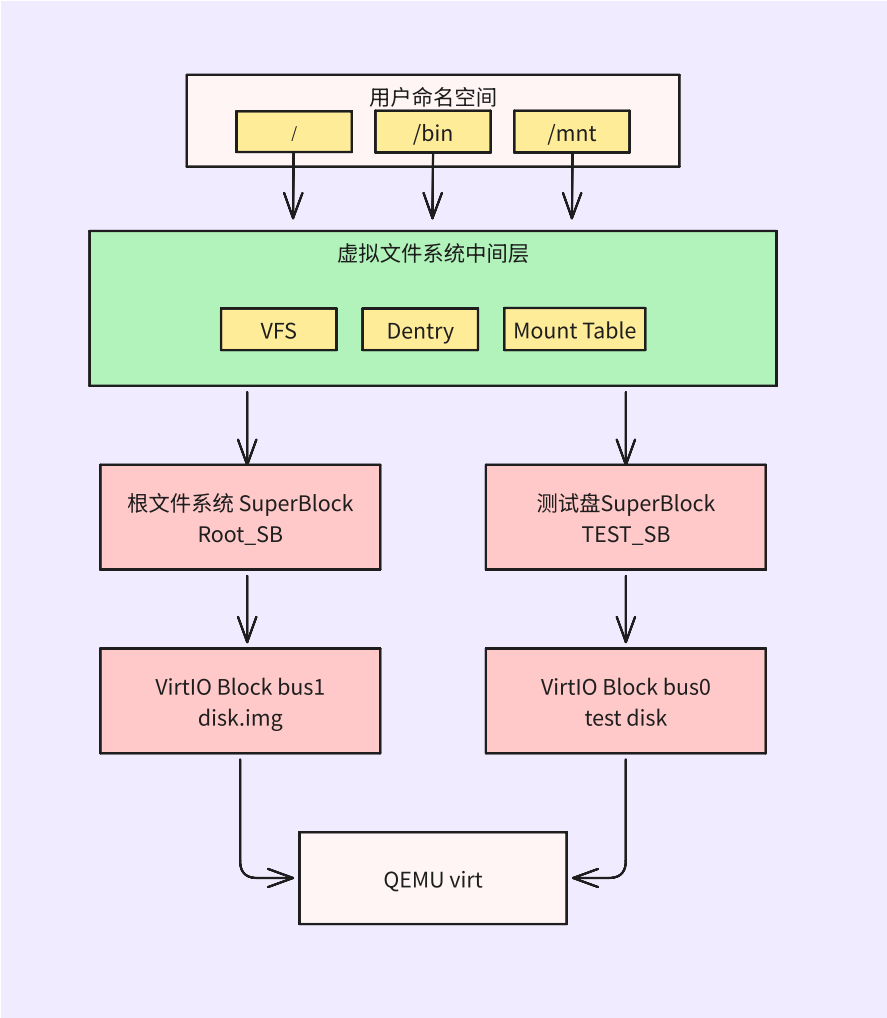


图 5.1　VFS 与双盘结构

5.5　挂载、卸载与一致性

挂载操作改变的是命名空间，而不是简单改变字符串前缀。执行 mount 系统调用时，内核先将目标路径解析为挂载点 Dentry，再为块设备创建新的 `VfsSuperBlock`，最后向全局挂载表加入挂载点与挂载根目录的对应关系。后续路径解析遇到该挂载点时，会转入新超级块根目录。卸载则需要删除相应挂载关系，并避免留下仍被打开文件或 cwd/root 引用的无效 Dentry。

一致性约束还体现在借用范围上。Ext4 访问通过 `with_fs` 闭包完成，以缩小对底层文件系统实例的可变访问范围；外层任务状态借用与 Ext4 闭包不应长时间重叠，否则可能造成 `RefCell` 借用失败或逻辑死锁。即使在单核环境下，文件系统仍然会和进程、内存、阻塞等待路径交错执行，因此需要清晰的对象边界。

5.6 特殊文件和 IPC

特殊文件和 IPC 说明了 VFS 不只是 Ext4 的封装。ScintillaOS 提供 `/proc/mounts`、`/proc/meminfo`、`/proc/<pid>/stat` 等 proc 风格接口，以及 `/dev/null`、`/dev/zero`、`/dev/urandom` 和 `/dev/rtc` 等设备文件。管道采用 4 KiB 环形缓冲区，支持阻塞和非阻塞读写。它们的底层实现各不相同，但都通过 `File` trait 进入 fd 表，因此系统调用层可以用同一套 read/write/close 语义处理普通文件、标准输入输出、管道和设备对象。

6 信号处理模块

6.1 信号发送与待处理状态

信号机制解决的是进程之间异步事件传递的问题。子进程退出、用户请求终止程序、非法内存访问或外部唤醒，都不能简单地等到目标进程下一次主动系统调用再处理。ScintillaOS 通过进程控制块中的待处理位图记录信号，`kill` 和 `tgkill` 根据 PID/TID 找到目标任务后，将信号置入目标的待处理集合。标准信号在位图模型中会合并；对于 `SIGKILL`、`SIGTERM` 等致命信号，内核同时更新终止状态。

仅记录信号还不够，因为目标任务可能正阻塞在 futex、wait、网络 I/O 或睡眠队列中。如果发送信号后不唤醒目标，目标就无法回到内核检查待处理状态。因此，信号发送路径与调度和阻塞队列存在直接依赖：发送端负责让目标具备再次运行的机会，真正的信号递送则推迟到目标即将返回用户态之前完成。这样可以避免在任意内核路径中强行改写用户现场。

6.2 信号递送时机

ScintillaOS 选择在陷阱返回用户态前递送信号。这个时机比较自然：内核已经拥有当前任务的 TrapFrame，可以安全修改用户态 PC、参数寄存器和用户栈；同时，任务尚未真正回到用户态，不会出现用户代码执行一半时内核突然插入处理函数的情况。递送逻辑会遍历待处理信号，选择未被信号阻塞掩码屏蔽的项；若信号具有用户处理函数，内核保存当前 TrapFrame，设置处理函数入口，传递信号编号并配置返回跳板；若使用默认动作，则按信号类型执行终止、停止、继续或忽略。

`sigreturn` 是用户处理函数返回内核的约定入口，用于恢复此前保存的执行上下文。因此，信号帧既要能让用户处理函数获得信号编号，又要保存旧掩码和被中断 PC。代码6.1摘自 `os/src/task/mod.rs:619` 附近的信号帧构造逻辑，展示了内核如何在用户栈中预留 `siginfo` 与 `ucontext` 空间。

代码 6.1　用户态信号帧的构造摘录

```
1 fn write_signal_frame(token: PageTable, sp: usize, sig: usize,
```

```
2                       old_mask: SignalFlags, interrupted_pc: usize) -> usize
                            {
3       let frame_size = SIGINFO_SIZE + UCONTEXT_SIZE;
4       let frame_base = sp.saturating_sub(frame_size) & !15;
5       for i in 0..frame_size { // 清零用户栈中的信号帧
6           *translated_refmut(token, (frame_base + i) as *mut u8) = 0;
7       }
8       *translated_refmut(token, frame_base as *mut i32) = sig as i32;
9       let ucontext = frame_base + SIGINFO_SIZE; // siginfo 之后保存 ucontext
10      *translated_refmut(token, (ucontext + UCONTEXT_SIGMASK_OFFSET) as *mut
            u64)
11          = flags_to_user_sigset(old_mask);
12      *translated_refmut(token, (ucontext + UCONTEXT_PC_OFFSET) as *mut
            usize)
13          = interrupted_pc;
14      frame_base
15  }
```

第 3 行向下扩展用户栈并按 16 字节对齐；第 4–6 行先清空帧内容，避免旧数据暴露给用户态；第 7–12 行填入信号编号、旧掩码和中断 PC。用户处理函数结束后，跳板执行 `sigreturn`，内核再根据这些信息恢复现场。该路径同时涉及用户地址写入、TrapFrame 修改和异常返回，必须和内存模块的用户指针检查、进程模块的阻塞唤醒保持一致。

6.3 信号处理的限制

信号处理的复杂性在于它跨越多个边界：发送阶段涉及 PID 查找和阻塞队列唤醒，递送阶段涉及用户栈布局和 TrapFrame 修改，返回阶段又依赖 `sigreturn` 恢复现场。ScintillaOS 已实现 `sigprocmask`、`rt_sigtimedwait` 等接口和基础递送路径，但复杂多进程信号同步、系统调用被中断后的返回值处理以及部分 LTP 场景仍属于风险区域。因此，信号模块的结论应区分“基础 kill/sigaction 路径可用”和“完整 POSIX 信号语义仍需持续验证”。

7　系统调用与用户态支持

7.1　系统调用接口

系统调用层是用户态程序观察内核能力的 ABI 边界。ScintillaOS 的系统调用入口位于 `os/src/syscall/mod.rs`，按 RISC-V Linux ABI 从寄存器接收最多 6 个参数，并将执行结果写回 TrapFrame 的返回寄存器。选择这一分层的原因在于，系统调用入口本身不应承载全部业务逻辑；它只负责解码调用号、读取参数并把请求转交给真正拥有状态的子模块。

ScintillaOS 覆盖的接口类别包括进程、文件、内存、信号、IPC、凭证、时间和网络等。正文以接口类别和代表性调用说明设计关系，完整清单通过表格呈现关键接口。代码7.1摘自 `os/src/syscall/mod.rs:180` 的分发表，展示了系统调用层如何把寄存器参数转换为 Rust 函数所需的整数、用户指针或结构体指针。

代码 7.1　文件系统系统调用分发摘录

```
let ret = match syscall_id {
    SYSCALL_OPENAT => sys_openat(
        args[0] as isize, args[1] as *const u8,
        args[2] as u32, args[3] as u32),
    SYSCALL_CLOSE => sys_close(args[0]),
    SYSCALL_GETDENTS64 =>
        sys_getdents64(args[0], args[1] as *mut u8, args[2]),
    SYSCALL_READ => sys_read(args[0], args[1] as *mut u8, args[2]),
    SYSCALL_WRITE => sys_write(args[0], args[1] as *mut u8, args[2]),
    SYSCALL_PIPE2 => sys_pipe2(args[0] as *mut usize, args[1]),
    _ => -38, // 未实现的系统调用返回 ENOSYS
};
```

例如，第 2–4 行将 `openat` 的目录 fd、用户态路径指针、打开标志和创建权限传给文件子系统；用户指针不会在分发层被直接解引用，而是在具体实现中通过页表转换和用户缓冲区检查访问。第 6–8 行显示读目录和普通读写虽然都经由 fd，但语义分别由 `getdents64` 和 `File` trait 处理。末行的 `-38` 对应 `ENOSYS`，用于让未实现接口以可诊断方式失败，而不是伪造成成功。

表 7.1　代表性系统调用类别

类别	代表性接口
进程	fork、clone、exec、waitpid、exit、brk、nanosleep
文件系统	openat、read、write、lseek、getdents64、mount、umount2、fsync
内存	mmap、munmap、mprotect、msync、mremap
信号	kill、sigaction、sigprocmask、sigreturn、rt_sigtimedwait
网络	socket、bind、listen、accept、connect、sendmsg、recvmsg

7.2　系统调用支持

Scintilla 实现的系统调用数量较多。我们按按接口归类，将实现的系统调用撰写在表7.2，并突出每类调用依赖的内核模块和当前支持状态。其中，“已实现”表示存在主要语义路径，“最小语义”表示接口为兼容特定程序提供受限行为，“占位或风险”则说明该调用不能作为完整兼容能力声明。

类别	代表接口（调用号）	支持级别	主要实现位置	说明
进程	fork(401)、clone(220)、execve(221)、waitpid(260)、exit(93)	已实现	`syscall/process.rs`、`task/task.rs`	fork 使用 COW；clone 根据标志共享资源。
文件 I/O	openat(56)、close(57)、read(63)、write(64)、lseek(62)	已实现	`syscall/fs.rs`、`fs/inode.rs`	通过 VFS/`File` trait 分发。
目录与元数据	getdents64(61)、fstat(80)、mkdirat(34)、renameat2(276)	已实现	`syscall/fs.rs`、`fs/vfs.rs`	依赖 Ext4 路径级操作。
描述符与 IPC	dup(23)、dup3(24)、fcntl(25)、pipe2(59)	已实现/部分	`syscall/fs.rs`、`fs/pipe.rs`	fcntl 仅覆盖当前测试所需命令。
内存	brk(214)、mmap(222)、munmap(215)、mprotect(226)、msync(227)	已实现/风险项	`syscall/process.rs`、`mm/memory_set.rs`	COW、保护页和复杂 LTP 场景仍需回归。
信号	kill(129)、sigaction(134)、sigprocmask(135)、sigreturn(139)	已实现/部分	`syscall/process.rs`、`task/signal.rs`	复杂用户态处理器返回路径仍是风险点。
时间与系统	nanosleep(101)、clock_gettime(113)、uname(160)、getrusage(165)	已实现/最小语义	`syscall/process.rs`	部分接口仅返回评测需要的字段。
凭证	getuid(174)、setuid(146)、getgroups(158)、umask(166)	已实现/部分	`syscall/process.rs`、`fs/vfs.rs`	权限语义受 Ext4 实现和测试覆盖限制。
网络	socket(198)、connect(203)、sendmsg(211)、recvmsg(212)	已实现	`net/syscall.rs`、`net/socket.rs`	支持 TCP/UDP 基础路径和环回。
共享内存	shmget(194)、shmat(196)、shmdt(197)	需复测	`syscall/process.rs`	不以本表作为完整 System V IPC 兼容性声明。
事件接口	epoll_create1(20)、eventfd2(290)、rseq(293)	占位/未实现	`syscall/mod.rs`	返回 `-ENOSYS` 或仅保留入口。

这张表需要和评测日志一起理解。增加一个调用号常量或 match 分支，并不等于该调用在错误码、阻塞语义和边界条件上都与 Linux 一致；真正的兼容性取决于它能否在对应模块中完成完整语义。例如 `mmap` 依赖虚拟内存和文件系统，`waitpid` 依赖进程生命周期，socket 调用则依赖网络端点和 fd 生命周期。

7.3 用户态运行时

用户态运行时为测试程序提供进入内核的最小支持。ScintillaOS 的 `user_lib` 提供 `_start` 入口、8 MiB 用户堆和系统调用包装，原始系统调用通过 RISC-V `ecall` 指令发起。选择静态链接策略，是为了降低早期 bring-up 阶段对动态加载器和共享库环境的依赖，使 shell、基础测试启动器、LTP 启动器和网络测试启动器能够先围绕内核 ABI 验证功能。用户态运行时虽然不属于内核核心，但它决定了测试程序如何构造参数、触发系统调用并解释返回值，因此也是 Linux 兼容链路的一部分。

8 网络模块

8.1 网络栈架构

网络模块要解决的是 Linux 用户程序如何通过 socket 接口访问 TCP/UDP 端点。BusyBox、iperf3 等程序只认识 fd 和 socket 系统调用，并不关心内核内部使用的是环回队列、smoltcp 还是 VirtIO 网卡。因此，ScintillaOS 将 TCP/UDP 端点包装为 socket fd，并复用文件描述符的创建、关闭和 I/O 生命周期管理方式。

网络子系统位于 `os/src/net`，包含本地环回路径，以及基于 smoltcp、VirtIO 网卡和 QEMU SLIRP 的外部网络路径。这样设计的原因在于，环回路径便于不依赖外部环境验证 socket 语义，VirtIO-Net 路径则用于接入 QEMU 网络后端。图中的数据路径展示了用户网络程序从 socket 系统调用进入内核后，如何被分流到 socket 表、环回队列或协议栈设备路径。

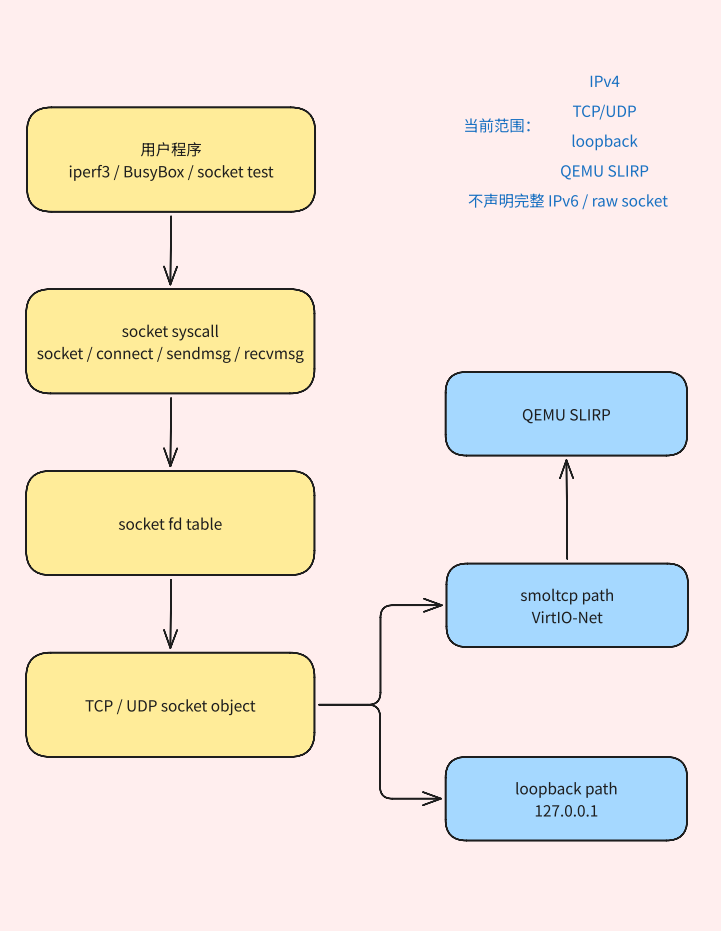


图 8.1 网络数据路径

8.2 套接字与阻塞 I/O

socket 系统调用创建套接字对象,随后通过 bind、listen、accept 和 connect 建立端点关系,数据收发则由 sendto/recvfrom 以及 sendmsg/recvmsg 完成。ScintillaOS 让 socket fd 和普通文件 fd 共享同一编号空间，是为了让用户程序能用统一方式持有、关闭和传递描述符；read/write 路径识别套接字描述符后，再转交网络子系统处理。

代码8.1为 os/src/net/syscall.rs:711--725 的关键摘录。该函数是网络接口进入内核后的第一个分流点：它检查协议族和类型，创建对应的内部 socket 对象，并将对象登记为可以返回给用户态的 fd。

代码 8.1 socket 系统调用的创建路径

```
pub fn sys_socket(domain: usize, type_: usize, _protocol: usize) -> isize {
    if domain != AF_INET { // 当前仅支持 IPv4
        return -97;
    }
    let st = match type_ & 0x0F {
        SOCK_STREAM => SocketType::Tcp,
        SOCK_DGRAM => SocketType::Udp,
        _ => return -22, // 不支持的 socket 类型返回 EINVAL
    };
    let nonblock = (type_ & SOCK_NONBLOCK) != 0; // 解析非阻塞标志
    let cloexec = (type_ & SOCK_CLOEXEC) != 0; // 解析执行时关闭标志
    create_socket_fd(st, nonblock, cloexec) // 创建对象并分配 fd
}
```

这里的关键不是返回一个整数 fd，而是让 fd 与套接字对象进入同一生命周期管理体系。后续 connect、sendmsg 和 recvmsg 都通过该 fd 找回同一个对象；close 时引用释放，端点状态才会被回收。对于当前不支持的协议族或 socket 类型，函数返回明确的 Linux errno，而不是分配一个无法使用的描述符。

阻塞 I/O 是网络模块与调度器之间的主要依赖。对于暂时无数据或连接尚未完成的操作，任务加入 NET_BLOCK_QUEUE 并让出 CPU；时钟中断和网络轮询推进协议栈后，满足条件的任务被唤醒。环回 TCP 使用监听表和双向缓冲区，UDP 使用报文队列，使 socket 功能能够在不依赖外部网络的情况下完成基础验证。

8.3 限制与适配范围

ScintillaOS 网络能力的边界需要单独说明。ScintillaOS 当前不支持 IPv6 和原始套接字，此外，在 iperf 与 netperf 测试用例中，TCP 链路依然存在接发信息不成功等问题。团队将在未来版本修复这些问题。

9 开发目标、计划与过程记录

ScintillaOS 的开发过程围绕“在 rCore 基础上扩展出可运行 Linux 用户程序的竞赛内核”展开。与普通课程实验相比，竞赛内核更强调跨模块语义一致性：一个系统调用能否通过测试，往往同时取决于进程生命周期、地址空间、文件描述符、路径解析和设备驱动等多条路径。因此，本章从目标、阶段计划和协作方式三个角度说明开发过程。

9.1 开发目标

ScintillaOS 的首要目标，是在 RISC-V QEMU virt 平台上提供稳定的 Linux ABI 兼容路径，使静态链接用户程序能够通过标准系统调用访问进程、文件、内存、信号和网络能力。选择 RISC-V、Ext4、VirtIO 和 QEMU 作为主要目标，是因为它们与 rCore 基础、比赛评测环境和可复现调试流程保持一致，能够让团队把精力集中在内核语义扩展上。

围绕这一目标，ScintillaOS 重点补齐了从 rCore 到竞赛内核之间的缺口：文件系统从单一实例扩展为支持挂载点和双盘的 VFS；进程模型从基础任务扩展为包含 fork、exec、wait、信号和文件继承的 Linux 风格对象；内存管理支持 COW、mmap 和按需分页；设备层接入 VirtIO 块设备和网络设备。当前范围以单核 QEMU 功能兼容为主，复杂 POSIX 语义和多核行为以实际测试覆盖为准。

9.2 阶段计划

表 9.1 开发阶段计划记录表

阶段	目标	主要工作	验证方式
阶段一	基础运行环境	启动、陷阱、进程切换、基础 syscall	basic 测试、串口日志
阶段二	文件系统迁移	Ext4、VFS、Dentry、双盘挂载	挂载与文件读写测试
阶段三	兼容性补齐	mmap、信号、fork/COW、pipe、proc/dev	BusyBox、Libctest、LTP 子集
阶段四	网络与性能	socket、环回、VirtIO-Net、iperf3、IOzone、Lmbench、Libcbench、lua	网络脚本、性能测试
阶段五	稳定化与文档	回归测试、日志整理、报告与答辩材料	最终提交日志与截图

9.3 完整开发过程描述

ScintillaOS 的开发由 rCore 原生，一步步适配测试用例的方式开发。开发初期的重点是保证基础执行稳定，并且能够执行二进制文件。因此，我们将 basic 与 busybox 测试用例所谓我们的测试目标。首先丰富了基础系统调用，并添加了用户栈，初始化页帧分配器和页表、建立陷阱入口、挂载根文件系统并装载 `/bin/initproc` 的过程。而针对评测，我们将 rCore 的原生文件系统替换成 ext4 文件系统并引入 VFS、Dentry、超级块和挂载表，把路径解析与具体文件系统实例分离，使 `/mnt` 后的路径能够切换到测试盘，同时仍然通过统一 fd 接口暴露给用户程序。

此时 ScintillaOS 已经可以通过 basic 与 busybox 的四种测试用例。随后我们转向 lmbench，lua，libctest 等对内核的运行稳定性，运行速度，语义完整度等有要求的多个测试用例。其中 fork 等进程有关操作因为反复报错占据了我们这部分开发的大部分时间。`fork` 需要复制用户现场和文件描述符表，但直接复制全部页面代价过高；ScintillaOS 因此学习 Linux 的思路，引入 COW，将父子进程初始页面共享为只读，并在写缺页时复制。信号、wait 和退出回收则进一步要求进程对象在退出后保留必要状态，直到父进程读取退出码。随后，我们尝试让操作系统适配网络模块，编写了有关系统调用，并在 ltp 等测试用例中进行深度评测。

最后，系统调用、测试适配进入稳定化阶段。此阶段我们不再只是增加调用

号，而是校准 ABI 参数、错误码、阻塞唤醒和边界条件，使用户程序能够通过更多测试用例。

9.4 团队分工协作与证据

我们在 Documents/Worklog.md 和 git 记录了详细记录了本团队的开发流程和开发分工，具体总结如下：

1. **雷雨泽（队长）：基础文件系统、Ext4/lwext4 迁移与项目统筹**。雷雨泽作为队长，主要负责早期文件系统迁移、基础文件系统调用适配和整体开发推进。其工作包括将文件系统从 rCore 基线迁移到 Ext4/lwext4，修复 openat、read/write、getdents64、dup/dup2、getcwd、unlink、chdir 等基础文件系统语义，并参与 main 与 MachineTest 分支的合并整理。相关证据包括 documents/worklog.md、documents/ext4 迁移记录 2026-04-22.md、documents/ext4 迁移具体变更内容.md、documents/VFS 与 Dentry 实现原理.md，以及 git 提交 e7ec1269、d4929035、bb33997f、26e0c1af、73617245 等。
2. **杜宇恒：内存管理、基础 syscall、网络系统与 LTP 推进**。杜宇恒主要负责 brk、mmap、munmap、times、uname、nanosleep 等系统调用实现，并承担网络子系统开发，包括 socket table/fd table 整合、socket、bind、listen、connect、sendto/recvfrom、sendmsg/recvmsg、setsockopt/getsockopt 等接口。后期还负责 LTP 可用用例筛选和测试流程推进。相关证据包括 documents/worklog.md、documents/ltp 测试指南.md、documents/ltp 子集测试状态.md、documents/netperf 测试指南.md、documents/iperf3 实施计划.md，以及 git 提交 5668a9b0、d0f74a2f、131ab1b5、0320fd35、6e4a5efd、7ea976be、0373875d、73e2f054 等。
3. **宋天琦：BusyBox/glibc/LoongArch 适配、官方评测链路与 benchmark 修复**。宋天琦主要负责 BusyBox、glibc BusyBox 和 LoongArch64 适配，包括 ELF 加载、用户栈、COW fork、LoongArch 启动与页表、BusyBox 测试脚本、/proc 虚拟文件系统、标准设备和 libc 兼容 syscall。后期主要负责官方 MachineTest 链路，包括官方 sdcard 双盘挂载、libctest/pthread/futex 语义、libcbench 线程退出与共享地址空间、lmbench/iozone 问题排查，以及从 MachineTest 向 main 的迁移记录。相关证据包括 documents/busybox_loongarch 适配具体更改.md、documents/busybox 系统调用实现计划.md、documents/libctest 开发文档.md、documents/测试记录.md、documents/MachineTest 迁移记录 2026-06-16.md、documents/MachineTest 近期改动迁移说明 2026-06-29.md，以及 git 提交 7780213d、e8e2a8de、4ab30d40、80315eff、e3bbe9cd、a741cce7、0948d279、f81fb9c9 等。

4. **协作方式与质量控制**。团队采用“文档记录 + git 小步提交 + 本地/官方双重验证”的协作方式。每一类功能开发或问题排查通常先在 `documents/worklog.md` 或专题文档中记录现象、根因、修复方案和验证结果，再通过 git commit 固化。对于复杂分支合并和官方评测链路，团队额外在 `documents/MachineTest` 迁移记录 `2026-06-16.md` 和 `documents/MachineTest` 近期改动迁移说明 `2026-06-29.md` 中记录迁移来源、保留内容、未迁移内容和验证情况，从而保证开发过程可追溯、可复现。

综上，雷雨泽、杜宇恒、宋天琦三人分别在文件系统与项目统筹、内存/网络/LTP、架构适配与评测链路等方向形成分工协作；`documents/worklog.md`、各专题文档以及 git commit 历史共同构成了开发过程和成员贡献的证据链。

10 Scintilla 技术问题及解决方案示例

ScintillaOS 的技术问题主要集中在跨模块语义不一致上。许多错误并不是某个函数单独失败，而是路径解析、页表权限、进程状态或等待队列之间的关系没有同步维护。本章选取几类代表性问题，说明问题现象、根因和解决思路。

10.1 问题记录

表 10.1 技术问题与解决方案记录表

问题类别	现象	根因分析	解决方案与验证
启动/陷阱	用户程序无法稳定返回内核	TrapFrame、页表切换和 `sret` 返回路径需要保持一致	统一陷阱入口，检查用户现场保存与恢复，并通过 initproc 启动日志验证
内存/COW	fork 后写入可能影响父子隔离	父子页表权限、共享页引用计数和写缺页复制必须同步维护	将可写页转为只读共享页，在写缺页时复制，并复测 fork/mmap 路径
VFS/Ext4	双盘挂载后路径可能落回根盘	用户命名空间路径与文件系统内部路径混淆	引入 Dentry、Mount 和超级块切换逻辑，通过 `/mnt` 路径访问测试盘验证
进程/信号	wait 不返回或信号处理后现场异常	Zombie 状态、父子唤醒和 sigreturn 用户栈布局存在跨模块依赖	区分退出与回收阶段，构造信号帧并在返回前恢复 TrapFrame

问题类别	现象	根因分析	解决方案与验证
网络/socket	accept/connect 或收发路径可能阻塞不醒	网络等待队列、轮询时机和 fd 生命周期需要保持一致	使用 socket 对象统一管理端点状态，通过环回与 iperf3 路径验证

10.2 案例一：VFS 与双盘路径解析

在 ScintillaOS 的早期内核版本中，文件子系统仅维护一个全局的 EXT4_FS 实例，所有路径解析都基于字符串拼接。例如，打开文件的流程为：

```
let full_path = normalize_path(cwd, path);
let inode = ext4_rs::generic_open(full_path);
```

进程的当前目录仅是一个 cwd: String，chdir 就是修改这个字符串。这种设计在小规模原型中勉强可用，但遇到 mount、多文件系统共存和精确目录语义时便立刻崩溃。

问题的根源在于字符串只能表达“位置”，无法承载“身份”与“边界”。字符串 /mnt/foo 不含任何关于它属于哪个超级块、对应哪块设备的信息。一旦在 /mnt 挂载了第二个 Ext4 文件系统，字符串解析仍会在根文件系统内寻找 /mnt/foo，完全无视本应切换到的测试盘。字符串看不见挂载点，也无法在 .. 操作时判断当前是否位于挂载根目录并跳回父文件系统。更隐蔽的是，将路径直接传给 generic_open 意味着解析逻辑与具体文件系统实现紧耦合，无论是添加挂载支持还是替换后端，都会引发连锁修改。因此，ScintillaOS 选择用携带 super_block、内部路径、父子关系的目录节点树（Dentry）来替代扁平字符串。

为此我们引入了 VFS 三大抽象。VfsSuperBlock 表示一个已挂载的文件系统实例：

```
pub struct VfsSuperBlock {
    pub fs_type: &'static str,
    pub fs_data: UPSafeCell<FsInner>,
}
```

每次 mount 生成一个 Arc<VfsSuperBlock>，所有底层操作通过 with_fs 闭包借用，避免长期持有内部状态。最关键的 Dentry 目录项则实现了双路径语义：

```
pub struct Dentry {
    pub name: String,
    pub path: String, // 对外统一路径，如 "/mnt/foo"
    pub fs_path: String, // 内部路径，如 "/foo"
```

```
    pub inode_num: u32,
    pub parent: Option<Arc<Dentry>>,
    pub super_block: Arc<VfsSuperBlock>,
    pub children: UPSafeCell<Option<Vec<Arc<Dentry>>>>,
}
```

path 用于 getcwd 等用户可见输出，fs_path 则剥离挂载前缀后传给底层文件系统 API，使后端完全不必感知自己被挂载在何处。进程控制块也相应升级：

```
pub struct TaskControlBlockInner {
    pub cwd: Arc<Dentry>, // 替换原来的 String
    pub root: Arc<Dentry>, // 支持 chroot
    // ...
}
```

同时我们添加了挂载点透明穿越。当解析 /mnt/foo 时，查找逻辑大致为：

```
fn vfs_lookup(parent: &Arc<Dentry>, name: &str) -> Option<Arc<Dentry>> {
    // 在 parent.children 中查找 name...
    if let Some(dentry) = found {
        // 检查挂载表
        if let Some(mount) = lookup_mount(&dentry) {
            return Some(mount.mnt_root.clone()); // 切换到挂载根
        }
        return Some(dentry);
    }
    // 未命中时通过 parent.super_block 加载目录项...
}
```

一旦命中挂载点，就返回挂载根的 dentry，其 super_block 已指向测试盘，fs_path 变为测试盘根目录 /。后续对子节点 foo 的查找会继承新超级块，形成正确的内部路径 /foo，而对外路径仍为 /mnt/foo。I/O 函数只需信任当前 dentry 绑定的超级块和内部路径：

```
fn read_at(dentry: &Dentry, buf: &mut [u8], offset: usize) -> Result<
    usize> {
    dentry.super_block.with_fs(|fs| fs.read_at(&dentry.fs_path, buf,
        offset))
}
```

反向的 .. 处理也需要感知挂载边界。vfs_parent 函数会检测当前 dentry 是否为某个挂载的根，若是则跳转到挂载点的父目录，从而从测试盘根正确返回到根文件系统的 /mnt 上级目录。

验证 /mnt 是否真正访问到测试盘的核心思路是“数据必须落在不同块设备上”。我们的一个做法是预置差异文件：在测试盘镜像中放入一个根盘没有的文件，挂载后执行 cat /mnt/exclusive 能读取到内容。还可以通过 stat 对比 st_dev 值，不同超级块必然返回不同设备号。最终，basic_runner 测试套件端到端覆盖了 mount、umount、chdir、getcwd 的全部场景，32 项测试全部通过，为迁移画上了可量化的句号。

从单 Ext4 实例到 VFS/Dentry/Mount 的迁移，本质上是用携带路由信息的目录节点图取代扁平字符串坐标。这一重构解耦了路径解析与具体实现，让应用上层只依赖构建的抽象接口而不依赖具体的后端实现，方便对 Ext4 文件系统的优化提升。

10.3 案例二：fork 与 COW 的一致性

ScintillaOS 最初的 fork 实现采用立即深拷贝策略：每次创建子进程时，遍历父进程地址空间中所有已映射的物理页，逐页分配新帧并调用 memcpy 完整复制。这种方案虽然简单且保证语义正确，但在 shell 等典型场景中——fork() 后几乎总紧接着 exec() 替换地址空间——拷贝的全部数据立刻被丢弃，造成了严重的 CPU 和内存浪费。更糟的是，多进程并发时物理内存瞬时翻倍，极易耗尽页帧导致系统崩溃，单次 fork 的巨大延迟也让交互体验急剧下降。为此，我们将内存管理机制改造为写时复制（Copy-on-Write，COW），期望在 fork 时实现零拷贝，仅在真正写入私有页时才付出复制代价，同时由引用计数保证物理页的安全回收。

COW 的设计围绕四个相互咬合的要素展开。fork 阶段，父子进程对私有可写页双向去除写权限，并通过 shared_clone() 共享同一物理页，只增加引用计数而不分配新帧。此时任意一方写入都会因权限缺失陷入 StorePageFault 异常。进入缺页处理后，系统根据物理页的引用计数决定动作：若计数大于 1，说明页面仍被其他进程引用，则分配新帧、完整复制数据，并将当前页表指向新帧、恢复写权限，旧的共享帧句柄被替换并递减计数；若计数已降为 1（另一进程已退出），则原地恢复写权限即可，无需任何复制。进程退出时，所有 FrameTracker 句柄通过 RAII 批量析构，递减引用计数，只有最后一个引用者才会真正归还物理页，实现了“零泄漏、无悬空”的精确回收。四要素以引用计数表为纽带，形成“权限修改触发陷入 → 计数判定复制 → 缺页完成隔离 → 退出归还物理帧”的闭环。

在实施过程中，我们曾遇到一个隐蔽的陷阱。msync(INVALIDATE) 操作会将页表项清零（V 位无效），但底层的地址翻译函数未检查有效位，对全零 PTE 仍返回物理页号 0。缺页处理函数误以为该页已映射并走入 COW 分支，却找不到合法的 FrameTracker，导致进程异常退出。子进程消亡后，父进程在管道上的 select 永久阻塞，系统呈现出“假死”现象。修正方法很简单：在地址翻译中严格校验 V 位，无效时直接返回 None，使上层能够正确区分“未映射”与“COW 保护”两种

状态。

改造完成后，我们通过差分观察法验证：fork 后子进程修改全局变量，父进程读取同名变量值不变，证明写入触发了真正复制；同时观察 fork 前后已分配帧数并未立即翻倍，确认零拷贝语义生效。在基础测试中，fork 与 clone 用例全部通过，lmbench 的进程与内存微基准也恢复正常。

10.4 案例三：系统调用边界条件

系统调用兼容的难点不只是找到调用号对应的处理函数，还要保持 ABI、错误码和边界条件一致。而在操作系统的内存管理模块中，mmap 系统调用的实现是极为核心且复杂的一环。它不仅要处理匿名映射以分配大块物理内存，还需负责文件映射，将磁盘内容透传至进程的虚拟地址空间。在 ScintillaOS 对 mmap 的实现和优化过程中就出现过一系列问题。

在 mmap 的功能逻辑初步连通后，又出现了内核崩溃，其原因是“哨兵值”的误用。当用户程序调用 mmap 且首参数传入空指针（即零值）时，其标准语义是请求内核自动寻找一块合适的空闲虚拟地址进行映射。然而，早期的内核逻辑将这个具有特殊语义的哨兵值进行了字面解析，直接将这段内存粗暴地映射到了虚拟地址的零地址处。在 RISC-V 的静态链接二进制可执行文件中，代码段往往就是从零地址或相近的极低地址区域开始加载的。这种字面解析导致新映射的区域直接覆盖了原有的用户态代码段，瞬间引发缺页异常或非法的指令执行错误。为了彻底修复这一边界缺陷，我们将选址逻辑进行了重构。修复后的机制为：当探查到起始地址参数为零时，系统会遍历当前地址空间的所有已映射区间，提取出最高的结束地址，并对其向上进行页对齐操作，从而在已有映射的上方安全地开辟出全新的区间。

```
let real_start = if start == 0 {
    let max_end: usize = inner.memory_set.max_end_vaddr().into();
    (max_end + PAGE_SIZE - 1) / PAGE_SIZE * PAGE_SIZE
} else {
    start
};
```

随着测试覆盖率的提升，LTP 测试套件中的高强度边界压力用例进一步击穿了内核的防御。在处理极端的高位地址或极大长度映射时，内核再次发生了致命的崩溃。经过详细的溯源与调试分析，问题的根源被定位在地址区间的运算方式上。内核在构建虚拟地址区间时，直接执行了裸加法运算以求取结束地址。在面对恶意的或极端的输入参数时，这一计算在无符号整数类型上发生了隐蔽的整数回绕，导致计算出的结束地址在数值上反而小于起始地址。随后，这个逻辑上倒挂的非法区间被传入底层区间构造函数，直接触发了严苛的断言机制。由于这是

在内核态直接触发的崩溃，意味着用户态的畸形输入能够毫无阻拦地打挂整个操作系统。针对这种严重的边界溢出漏洞，我们的解决思路是在所有涉及地址与长度相加的逻辑链路中，强制引入溢出检查机制。通过替换为安全的加法操作，一旦探测到回绕风险，系统会立刻阻断映射流程并向用户态返回无效参数的错误码，从而筑牢了抵御非法输入的第一道防线。

通过对传参宽度、哨兵值解析以及整数溢出等问题的逐一击破，目前的 mmap 实现已能顺利通过基础测试以及部分高强度的 LTP 压力用例。

11 构建、评测与结果

11.1 构建与运行方法

ScintillaOS 根目录的 `Makefile` 提供如下常用目标：

- `make all`：生成 RISC-V、LoongArch 内核及根文件系统镜像；
- `make kernel ARCH=riscv64`：仅编译 RISC-V 内核；
- `make fs-img ARCH=riscv64`：构建包含用户程序的 Ext4 镜像；
- `make run ARCH=riscv64`：单盘开发模式运行；
- `make run-dual ARCH=riscv64 TEST_DISK=<测试盘>`：双盘方式模拟评测环境。

11.2 测试框架

ScintillaOS 运行在官方提供的 docker 环境中，由 `initproc` 在双盘模式下挂载测试盘，扫描并运行官方测试盘中的 `testcode.sh` 测试脚本。测试输出使用统一的开始/结束标记，外部裁判脚本据此提取测试组结果。

12 总结与展望

12.1 当前工作情况

ScintillaOS 完成了从基础教学内核向竞赛测试环境的扩展：通过 VFS 和 lwext4 构建可挂载的 Ext4 文件系统；通过双盘结构适配评测流程；通过进程、内存、信号与 Linux ABI 系统调用支撑静态用户程序；并接入多类测试套件。Ext4 迁移、Dentry 缓存、按需分页与 COW、socket 文件描述符统一管理是系统实现中的关键工作。

12.2 经验总结

在 ScintillaOS 的开发中，我们遇到了不少问题与挑战，也由此积累了许多开发经验。

首先，在文件系统从单一 Ext4 实例迁移到 VFS/Dentry/Mount 架构时，难点是把路径解析与底层实现解耦，我们为每个目录项同时维护命名空间路径与文件系统内部路径，才实现了挂载点的透明穿越。其次，在 fork 写时复制中，立即深拷贝会浪费内存、直接共享又会使父子互相踩踏，我们通过物理页引用计数、父子双向去写权限和写缺页复制来兼顾性能与隔离。

这些问题的解决，让我们在系统架构设计和问题解决能力方面获得了显著提升，我们也意识到系统实现过程中，调试是最为关键的一步，我们也由此提升了调试优化系统设计的能力，团队协作愈发成熟。

12.3 未来展望

在后续的工作中，我们计划继续优化 ScintillaOS 的功能和性能，主要包括以下方面：

第一，重新回归 COW、信号处理返回和阻塞/唤醒路径；第二，补齐或明确标注 LTP 中依赖较强的接口，争取通过更多的测试用例；第三，完善网络协议栈，优化网络通信问题，支持更多网络功能，提升网络并发性稳定性；第四，优化 VFS Dentry 缓存策略和回收机制。

附录 A　AI 辅助使用说明

ScintillaOS 在开发过程中使用了生成式 AI、AI Agent 和智能 IDE 辅助工具。主要使用的工具包括 deepseek v4 pro、Codex 类 AI Agent，以及编辑器中的智能补全能力。AI 工具并不只用于文档润色，也参与了代码阅读、问题定位、局部实现草案生成、补丁修改建议、测试日志分析和报告整理等工作。团队在使用 AI 的过程中保留了主要交流记录，并通过 `documents/worklog.md`、各专题文档以及 git 提交历史记录具体功能的实现过程、问题原因和验证结果。

在系统设计层面，ScintillaOS 的总体技术路线由团队确定，即在 rCore 教学内核的基础上继续扩展，面向 QEMU 官方评测环境实现 Linux ABI 兼容能力。系统采用 main 分支作为主要开发分支，MachineTest 分支作为官方评测分支；这一分支策略、双盘评测模型、VFS/Dentry/Mount 架构、Ext4/lwext4 迁移、进程与地址空间组织、官方测试链路安排等关键设计，均由团队成员结合比赛要求和本项目实际情况确定。AI 在这一过程中主要帮助整理方案表述、归纳模块依赖关系、提示可能的风险点，但最终设计取舍由团队成员完成。

在代码实现层面，AI 参与了较多局部开发工作。例如，在文件系统、进程管理、内存管理、futex、信号、测试 runner、评分输出格式、README 和部分文档整理中，AI 曾根据已有代码生成局部补丁、提出修改方案或帮助重构代码片段。部分代码最初由 AI 给出草案，再由队员根据项目架构进行人工修改、合并和验证；也有部分代码由队员先写出主体，再使用 AI 辅助检查边界条件、错误码、日志输出和文档说明。因此，最终代码并非完全人工逐字编写，也不是 AI 独立生成后直接提交，而是经过队员理解、筛选、改写和测试后的结果。

在测试和调试层面，AI 对项目帮助较大。团队曾使用 AI 辅助分析 QEMU 串口输出、panic 日志、官方 judge 脚本行为、LTP 输出格式、libctest 超时原因、libcbench 线程退出卡死、lmbench 和 iozone 的性能与文件系统问题等。相关过程可以在 `documents/libctest 开发文档.md`、`documents/ltp 测试指南.md`、`documents/ltp 子集测试状态.md`、`documents/测试记录.md`、`documents/MachineTest 迁移记录 2026-06-16.md` 和 `documents/MachineTest 近期改动迁移说明 2026-06-29.md` 中看到。AI 在这些场景中提供了排查方向和候选解释，但是否采纳、如何修改、是否回退以及是否进入最终评测版本，均由团队通过本地 Docker/QEMU 测试和官方评测结果判断。

AI 生成内容并不总是正确。开发过程中曾出现过 AI 对官方评分脚本理解不完整、对系统调用语义判断过于简化、忽略 LoongArch 与 RISC-V 架构差异、建议的超时策略不适合最终评测、生成的代码与现有借用关系或锁顺序冲突等问题。团队发现这些问题后，会通过阅读源码、比对 Linux 语义、重新运行测试、查看

git diff 和分析日志来修正。对于可能影响核心语义的修改，例如 COW、页表回收、robust futex、clear_child_tid、VFS 路径解析、pipe 阻塞唤醒和 socket fd 管理，团队不会只依据 AI 输出判断正确性，而是结合测试结果和代码审查决定是否保留。

在文档整理阶段，AI 也参与了较多工作。团队使用 AI 根据已有源码、开发日志和专题文档整理报告内容，包括系统总体架构、模块边界、团队分工、libctest 开发说明、README、总结与展望等。文档中的技术结论主要来自仓库代码、git 记录、官方评测结果和本地测试日志；AI 的作用是帮助组织语言、提炼结构和减少遗漏。若 AI 生成的表述与实际代码或测试结果不一致，团队以实际代码和测试结果为准，并对文档进行人工修正。

最终提交版本的设计逻辑和技术细节由团队掌握。团队成员能够解释 ScintillaOS 的启动流程、系统调用分发、进程和线程模型、地址空间管理、COW 与按需分页、VFS/Dentry/Mount 结构、Ext4 双盘访问、futex 与信号处理、socket fd 统一管理以及官方评测链路。AI 工具在项目中承担的是辅助开发、辅助分析和辅助表达的角色，而不是替代团队完成方案设计、核心理解、最终调试和提交决策。对于最终代码中的 AI 参与部分，团队已经通过人工修改、构建检查、本地运行和官方评测进行了确认，并能够在需要时独立继续修改和调试。

对于任何由 AI 协助编写或修改的内容，团队成员均应完成以下确认：第一，对照实际源代码逐行审阅，确认函数名、数据结构和控制流未被误述；第二，在目标架构和 QEMU 环境中重新编译、运行对应测试；第三，在 `documents/worklog.md` 或版本控制记录中保留功能变更、责任人和验证证据。特别是内核中的 `unsafe`、页表、用户指针、并发控制和设备 I/O 路径，不能仅依据 AI 生成文本判断正确性。

本报告中的设计取舍、支持状态和已知限制以仓库代码、提交记录、测试日志及小组人工审核为准。若最终提交版本与报告中的代码摘录不一致，应以最终代码与重新生成的测试结果更新本报告。

附录 B　参考资料

1. RISC-V International，*The RISC-V Instruction Set Manual*。
2. lwext4 文档与源码。